# PIXEL PACT:AI-POWERED DIGITAL ART WITH BLOCKCHAIN OWNERSHIP

**PROJECT REPORT**

*Submitted to the Department of computer science,*

**MALABAR COLLEGE OF ARTS AND SCIENCE, MOODAI**

*In partial fulfilment of the Requirement for the Award of*

**DEGREE OF BACHELOR OF COMPUTER APPLICATION UNIVERSITY OF CALICUT 2023 – 2026**

*Submitted by*

**NAME : MUHAMMED NIYAD T**
**Reg No : MXAXBCA041**

***Under the Guidance of:*** **Guide Name: Mrs. PRIYANKA.K.P**

**Designation: HOD Dept of Computer Science**

**DEPARTMENT OF COMPUTER SCIENCE MALABAR**

**COLLEGE OF ARTS AND SCIENCE**

MOODADI NORTH – 673307

# PIXEL PACT:AI-POWERED DIGITAL ART WITH BLOCKCHAIN OWNERSHIP

**PROJECT REPORT**

*Submitted to the Department of computer science,*

**MALABAR COLLEGE OF ARTS AND SCIENCE, MOODAI**

*In partial fulfilment of the Requirement for the Award of*

**DEGREE OF BACHELOR OF COMPUTER APPLICATION UNIVERSITY OF CALICUT 2023 – 2026**

*Submitted by*

**NAME : MUHAMMED NIYAD T**
**Reg No : MXAXBCA041**

***Under the Guidance of:*** **Guide Name: Mrs. PRIYANKA.K.P**

**Designation: HOD Dept of Computer Science**

**DEPARTMENT OF COMPUTER SCIENCE MALABAR**

**COLLEGE OF ARTS AND SCIENCE**

MOODADI NORTH – 673307

# MALABAR COLLEGE OF ARTS AND SCIENCE

MOODADI – 673307

PROJECT REPORT ON

# PIXEL PACT

*Submitted for the Partial fulfilment of for the award of the degree*

**BACHELOR OF COMPUTER APPLICATION**

**NAME Reg No: MXAXBCA041**

# DECLARATION

I **MUHAMMED NIYAD T** bearing the Register **No MXAXBCA041** studying in the final semester of Bachelor of Computer application at Malabar College of Arts and Science Moodadi North, hereby declare that this project work entitled "**PIXEL PACT**" which is being submitted by them in the partial fulfilment for the award of the degree of Bachelor of Computer Science, from University of Calicut, Thenjippalam is an authentic record of me carried out during the academic year 2025-2026, under the guidance of Mrs. **PRIYANKA.K.P**, H.O.D of Computer Science, MCAS Moodadi North.

(Signature of the candidates)

**NAME: MUHAMMED NIYAD**

**Reg.no: MXAXBCA041**

I certified that the declaration made above by the candidate is true

(Signature of the guide)
**MRS.PRIYANKA.K.P**
**H.O.D. in Computer Science**

# ACKNOWLEDGEMENT

# CONTENTS


| CHAPTER | TITLE | PAGE.NO |
|---|---|---|
| i | ABSTRACT | |
| ii | LIST OF FIGURE | |
| iii | LIST OF TABLE | |
| iv | LIST OF ABBRIVIATION | |
| **1** | **INTRODUCTION** | **1** |
| 1.1 | PURPOSE OF THE PROJECT | 3 |
| 1.2 | SCOPE OF THE PROJECT | 3 |
| **2** | **LITERATURE REVIEW** | 4 |
| 2.1 | LITERATURE REVIEW | 5 |
| 2.2 | References | 5 |
| **3** | **SYSTEM ANALISYS** | **6** |
| 3.1 | INTRODUCTION | 7 |
| 3.2 | STUDY PHASE | 7 |
| 3.3 | EXISTING SYSTEM | 8 |
| 3.4 | LIMITATIONS OF THE EXISTING SYSTEM | 8 |
| 3.5 | PROPOSED SYSTEM | 9 |

| | | |
|---|---|---|
| 3.6 | OBJECTIVES OF THE DETAILED STUDY | 9 |
| 3.7 | Benefits of the Proposed System (Revised) | 10 |
| **3.8** | **FEASIBILITY STUDY** | **11** |
| 3.8.1 | TECHNICAL FEASIBILITY | 12 |
| 3.8.2 | OPERATIONAL FEASIBILITY | 12 |
| 3.8.3 | ECONOMIC FEASIBILITY | 13 |
| 3.8.4 | TIME FEASIBILITY | 13 |
| 3.8.5 | CONCLUSION | 14 |
| **3.9** | **MODULE DISCRIPTION** | **14** |
| 3.9.1 | ADMIN | 14 |
| 3.9.2 | USER | 14 |
| 3.9.3 | ARTIST | 15 |
| 3.10 | PROJECT SHEDULING | 16 |
| **3.11** | **SPRINT PLANNING AND MANAGEMENT** | **17** |
| 3.11.1 | **SPRINT OVERVIEW** | 17 |
| 3.11.2 | **SPRINT OBJECTIVES** | 17 |
| 3.11.3 | **SPRINT DETAILS** | 18 |
| 3.11.4 | ***SPRINT REVIEW*** | 18 |
| 3.11.5 | ***SPRINT RETROSPECTIVE*** | 18 |

| | | |
|---|---|---|
| 3.12 | USER STORIES | 19 |
| **4** | **SYSTEM REQUIREMENT SPECIFICATION** | **20** |
| 4.1 | INTRODUCTION | 21 |
| 4.2 | SOFTWARE SPECIFICATION | 22 |
| 4.3 | HARDWARE SPECIFICATION | 24 |
| 4.4 | TECHNOLOGY SPECIFICATION | 24 |
| **5** | **DESIGN PHASE** | **26** |
| 5.1 | INTRODUCTION | 27 |
| 5.2 | ARCHITECTURAL DESIGN | 28 |
| | | |
| | | |
| | | |
| | | |
| | | |
| | | |
| | | |
| | | |
| | | |
| | | |

| | | |
|---|---|---|
| | | |
| | | |
| | | |
| | | |

# ABSTRACT

In the expanding field of digital art, there is a growing need for a platform that enables seamless, personalized collaboration between users and artists while ensuring strong protection against image duplication and unauthorized copying. PixelPact is a digital art creation and customization system that allows users to generate artwork using AI-driven image generation or upload their own concepts for refinement. Users can provide prompts, stylistic preferences, and visual references, enabling the integrated AI engine to produce initial art concepts quickly and efficiently.

Alongside AI-generated outputs, users can request professional artists on the platform to refine, enhance, or fully recreate artwork according to user requirements. To safeguard originality, PixelPact integrates blockchain-based digital ownership recording. Each finalized artwork is registered on the blockchain, creating an immutable proof of authorship and ensuring images cannot be duplicated, altered, or falsely claimed without detection. This protects both users and artists from plagiarism, unauthorized reproduction, and intellectual property disputes.

By combining AI automation, creative customization, artist expertise, and decentralized ownership protection, PixelPact delivers a next-generation platform for secure, personalized, and trustworthy digital art creation.

# LIST OF FIGURES


| | | | |
|---|---|---|---|
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |

# LIST OF TABLES


| | | | |
|---|---|---|---|
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |

# LIST OF ABBREVIATIONS

| | | |
|---|---|---|
| | | |
| | | |
| | | |
| | | |
| | | |
| | | |
| | | |
| | | |
| | | |

# 1
# INTRODUCTION

# INTRODUCTION

In today's rapidly evolving digital world, the demand for unique and innovative digital art continues to grow, yet artists still face challenges in showcasing their work, securing commissions, and protecting their intellectual property. PixelPact emerges as a cutting-edge solution to address these challenges by offering a blockchain-enhanced collaborative digital art platform that blends creative expression with secure, transparent interactions. The platform allows users to generate artwork through AI-driven tools or upload concept ideas, while artists can refine, enhance, or fully recreate these concepts based on client requirements.

To ensure authenticity and safeguard originality, PixelPact integrates blockchain technology to register finalized artworks, ownership records, and creative contributions in an immutable ledger. This provides strong protection against image duplication, unauthorized copying, and plagiarism, giving both artists and clients confidence in the integrity of their digital assets. Smart contracts further enhance security by automating essential processes such as ownership verification and permission handling, reducing the risk of disputes and ensuring fairness without manual intervention.

The mobile application enhances accessibility, enabling users and artists to communicate, review, and manage projects anytime and anywhere. AI-driven features offer intelligent suggestions, enhancements, and previews, supporting creativity and experimentation while maintaining artistic originality. By combining secure technologies, collaborative tools, and an intuitive user experience, PixelPact empowers artists to monetize their creativity more effectively while fostering a vibrant, trustworthy digital art community that bridges the gap between artistic talent and market demand.

## 1.1. PURPOSE OF THE PROJECT

The primary purpose of the **PixelPact** project is to build a secure and transparent digital marketplace that connects digital artists with art enthusiasts. Traditional online art platforms often face issues such as lack of credibility, unfair pricing, plagiarism, and unauthorized duplication of artwork. PixelPact addresses these challenges by integrating a **blockchain-based artwork verification system** that ensures originality, prevents duplication, and provides immutable proof of ownership for every digital creation.

The platform enhances the creative experience through features like **AI-assisted concept generation** and **direct commission requests**, enabling users and artists to collaborate smoothly in a fair and structured environment. PixelPact also supports seamless transactions through a simple and reliable payment system designed for user convenience.

Ultimately, the project aims to create a next-generation digital art marketplace that promotes authenticity, protects creative rights, and fosters innovation within the digital art ecosystem.

## 1.2. SCOPE OF THE PROJECT

PixelPact aims to revolutionize the digital art ecosystem by providing a secure, collaborative, and blockchain-powered platform for artists and art enthusiasts. The project facilitates seamless interaction between creators and clients through features such as art concept generation direct commissioning, all while ensuring transparency and trust via smart contracts. It also incorporates a mobile application, making art collaboration and marketplace access more flexible and user-friendly. The scope of PixelPact extends to supporting secure transactions, preserving intellectual property rights, and offering AI-assisted suggestions for art enhancement. Furthermore, it provides a structured environment for both amateur and professional artists to showcase their talent, receive feedback, and monetize their work,

# 2

# LITERATURE REVIEW

## 2.1. LITERRATURE RIVIEW

In recent years, digital art marketplaces have rapidly evolved, yet most centralized platforms continue to face challenges such as copyright infringement, lack of transparency, and unreliable content verification. Researchers have highlighted that blockchain technology can provide effective solutions by ensuring immutable records and verified ownership of digital assets [1]. Studies on decentralized systems emphasize that blockchain is highly valuable for maintaining authenticity and preventing duplication or unauthorized reuse of creative works [2]. Additionally, the integration of Artificial Intelligence (AI) into creative platforms enhances personalization and innovation, enabling users to generate or refine digital art concepts efficiently [3]. Prior research on NFT-based networks and decentralized art repositories demonstrates that blockchain strengthens provenance tracking and protects artists' intellectual property rights [4]. Building upon these findings, PixelPact incorporates blockchain-based artwork verification and AI-driven tools to create a transparent, user-centered ecosystem that promotes authenticity, credibility, and creative freedom in digital artistry.

## 2.2. References

1. Nakamoto, S. (2008). Bitcoin: A Peer-to-Peer Electronic Cash System.
2. Buterin, V. (2014). A Next-Generation Smart Contract and Decentralized Application Platform. Ethereum White Paper.
3. Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press.
4. Dowling, M. (2022). NFTs and the Future of Digital Art: Blockchain-Based Provenance and Ownership. Journal of Business Research, 142, 758–769.

# 3

# SYSTEM ANALYSIS

## 3.1.INTRODUTION

System Analysis is a crucial phase in the software development life cycle (SDLC) that focuses on understanding, examining, and defining the functional and non-functional requirements of a system. In this phase, the existing system (if any) is studied to identify its limitations, inefficiencies, and challenges, followed by designing a new system that effectively addresses those issues. For PixelPact, system analysis involves a detailed examination of how digital art platforms currently operate, the problems faced by users and artists—such as lack of transparency, insecure transactions, and limited communication—and how a blockchain-based solution can overcome these barriers. This stage helps to clearly define user roles, system modules, data flow, and the interaction between components to ensure the final design aligns with real-world needs. Through careful analysis, PixelPact aims to build a robust, secure, and user-friendly platform that enhances trust, creativity, and collaboration between digital artists and clients.

## 3.2. STUDY PHASE

The PixelPact project focuses on thoroughly analyzing the current scenario of digital art platforms, identifying their shortcomings, and formulating a structured solution that enhances transparency, security, and authenticity. This phase highlights key challenges such as copyright infringement, duplication of artwork, lack of proper verification, and limited communication between artists and users. By incorporating a **blockchain-based artwork verification system**, PixelPact ensures secure and immutable proof of ownership while preventing unauthorized copying or misuse of digital art. The analysis also helps in understanding the overall workflow, system requirements, and user expectations, enabling the development of a platform that effectively addresses existing limitations and provides a reliable, user-centered experience.

## 3.3. EXISTING SYSTEM

Most current digital art platforms primarily function as portfolio showcases or direct commission websites. These systems lack transparency in transactions, pricing clarity, and proper verification of artist authenticity. Users often face issues like overpricing, delayed delivery, fraud, or plagiarism. There is no reliable mechanism to ensure that payments are secure or that the ownership of digital artworks is recorded and protected. Furthermore, communication between clients and artists is often fragmented, with no unified environment for collaboration or negotiation.

## 3.4.LIMITATIONS OF THE EXISTING SYSTEM

- Lack of trust and transparency between digital artists and users on traditional platforms.
- No reliable mechanism to verify the authenticity of digital artworks or prevent duplication and art theft.
- Insecure or unreliable payment processes, leading to disputes and risk of fraud.
- Limited communication channels for artist-client interaction, making collaboration inefficient.
- No proper system for feedback or progress tracking, reducing clarity during commissions.
- Absence of a permanent, tamper-proof record of artwork ownership, leading to misuse or unauthorized distribution.

## 3.5. PROPOSED SYSTEM

The PixelPact system introduces a secure and user-centered platform designed for digital art collaboration, creation, and commissioning. Instead of relying on traditional centralized models, PixelPact integrates a blockchain-based artwork verification system to ensure authenticity, prevent duplication, and maintain immutable ownership records for every digital artwork. Users can generate or refine art concepts using AI tools and directly commission artists of their choice through a streamlined and transparent process.

The platform includes features such as AI-assisted image generation, artist verification, secure payment handling (non-blockchain), real-time chat, and rating systems to support smooth communication and build trust between artists and clients. By combining advanced verification methods with user-friendly interaction tools, PixelPact offers a reliable, creative, and secure environment for modern digital art engagement.

## 3.6. OBJECTIVES OF THE DETAILED STUDY

- To analyze user requirements and system functionalities needed for a modern digital art collaboration platform.
- To compare and evaluate existing digital art systems and identify gaps such as lack of authenticity verification, poor communication, and copyright issues.
- To design a system structure that integrates blockchain for artwork authenticity, duplication prevention, and ownership verification.
- To establish clear modules for admin, user, and artist roles to ensure organized workflow and role-based access.
- To ensure scalability, reliability, and user-friendly operation so the platform can grow with future enhancements like advanced AI tools.

## 3.7. BENEFITS OF THE PROPOSED SYSTEM (REVISED)

- Transparency: Artwork authenticity and ownership are verifiable through blockchain records.
- Security: Blockchain-backed verification prevents art theft, duplication, and unauthorized usage.
- Efficiency: Streamlined communication and direct commission system make collaboration smooth.
- Fairness: Artists gain proper recognition and protection for their work through verified records.
- Scalability: The system can be expanded to support global art sharing, advanced AI features, and additional creative tools.

## 3.8.FEASIBILITY STUDY

A feasibility study is a preliminary investigation undertaken to determine and document a project's viability. The results of this study are used to decide whether to proceed with the project. If approved, it helps ascertain the likelihood of the project's success before actual development begins. It involves analyzing possible alternative solutions to a problem and recommending the most suitable one. The feasibility study provides a conceptual idea of the new system, specifying what tasks will be automated, what inputs and outputs are required, and which processes remain manual. During this phase, all alternatives are discussed and evaluated based on technical, operational, and economic aspects. A cost-benefit analysis is often carried out to compare the cost of implementation with the expected benefits, which are usually more qualitative than quantitative.

**A feasibility study is necessary when:**

- The current system no longer meets requirements.
- Technological advancements make the current system obsolete.
- The organization is expanding and needs better workload management.
- Users or customers express dissatisfaction with performance or quality.
- Competitors gain an advantage through improved digital systems

Thus, the feasibility study helps determine whether the proposed PixelPact system can be developed with available resources, time, and technology

## 3.8.1.TECHNICAL FEASIBILITY

The PixelPact system is technically feasible as it is developed using modern, scalable, and reliable technologies. The web frontend is built using the MERN stack, where React.js is used to create a dynamic, responsive, and interactive user interface. The backend services are implemented using Node.js and Express.js, ensuring efficient handling of user authentication, artwork management, payments, and other system operations.

The mobile application is developed using Flutter, enabling cross-platform deployment across Android and iOS devices from a single codebase. For AI-based artwork generation and processing, Python FastAPI is used to provide high-performance API services, ensuring fast and scalable AI integration with the main system.

MongoDB is used as the primary NoSQL database for efficient storage and retrieval of user data, artworks, cart details, chat messages, and transaction records. Blockchain integration ensures secure and immutable storage of artwork ownership details, preventing duplication and maintaining authenticity. Development tools such as Visual Studio Code, Android Studio, and Postman are utilized for coding, debugging, and API testing. Version control is maintained using Git to ensure systematic code management. The hardware requirements for running PixelPact are minimal (Intel i3 or above, 4GB RAM, and 512GB storage), making the system

deployable on standard computing environments. Hence, the proposed solution is technically feasible and practically implementable.

## 3.8.2.OPERATIONAL FEASIBILITY

Operational feasibility determines how effectively the new system will function once implemented. PixelPact is designed with a user-friendly interface to ensure easy accessibility for both artists and clients. Features like AI-generated art concepts,real-time chat, payment management, and blockchain verification are straightforward and practical for users.

**The system benefits all stakeholders:**

- **Users** can securely commission artworks, post concepts, and choose artists through transparent bidding.
- **Artists** gain exposure, fair payment, and protection against plagiarism.
- **Admins** can efficiently manage users, verify artists, and monitor activities.

Training requirements are minimal, as the platform follows standard web and mobile usability practices. Thus, operational adoption and day-to-day functionality are both highly feasible.

## 3.8.3.ECONOMIC FEASIBILITY

Economic feasibility evaluates whether the PixelPact project is financially practical and sustainable. PixelPact primarily utilizes open-source technologies such as the MERN stack (MongoDB, Express.js, React.js, Node.js), Flutter, and Python FastAPI, which significantly reduce software licensing costs. Since these technologies are free and widely supported, the overall development expenditure remains cost-effective.

The primary expenses involve cloud hosting services, domain registration, blockchain integration for secure ownership verification, and initial development and maintenance efforts. Additional costs may include server scaling and AI model optimization as the platform grows.

In the long term, PixelPact offers strong economic benefits through secure, automated, and decentralized digital art transactions. By minimizing intermediaries and reducing fraud risks through blockchain-based ownership verification, operational costs are lowered while trust is increased. The platform also has strong revenue-generation potential through commission fees on transactions, premium subscriptions, featured artist promotions, and AI-based premium services.

Therefore, the economic feasibility of PixelPact is strong, with high potential for long-term sustainability and return on investment (ROI).

### 3.8.4.TIME FEASIBILITY

Time feasibility examines whether the project can be completed within a reasonable timeframe. The PixelPact project follows a structured development approach divided into phases—analysis, design, development, testing, and deployment. With efficient task allocation and use of modern frameworks, the system can be completed within the planned schedule without compromising quality.

The use of ready-made libraries, blockchain APIs, and cloud hosting services further speeds up implementation. Therefore, PixelPact is time-feasible and achievable within a standard academic or professional development duration.

### 3.8.5.CONCLUSION

The feasibility analysis of PixelPact demonstrates that the project is **technically, operationally, economically, and temporally feasible.** With the combination of blockchain technology, secure architecture, and user-oriented design, the system is practical to implement and promises significant benefits for digital artists, clients, and administrators. This confirms that PixelPact is a sustainable and valuable solution for revolutionizing the digital art marketplace.

## 3.9.MODULE DISCRIPTION

### 3.9.1.Admin

This module provides administrative control over the system. Admin can verify artists, monitor users, manage complaints, block/unblock accounts, and oversee platform activities.

- **Login** – Admin logs into the system using secure credentials.
- **View User** – Admin can see the list of registered users.
- **Verify Artist** – Admin checks and approves genuine artist accounts.
- **View Complaints and Send Reply** – Admin views user/artist complaints and responds to them.
- **View Art Ratings** – Admin monitors ratings given to artworks and artists.
- **Block/Unblock Artist** – Admin can block artists who violate rules or unblock them if needed.

### 3.9.2.User

This module handles registration, login, and profile management for Users, Artists, and Admin. It ensures secure authentication and role-based access control. Users can update personal details, manage accounts, and access features based on their role.

- **Register** – User creates a new account in PixelPact.
- **Login** – User logs in using email and password.
- **Update Profile** – User edits personal details like name, photo, and contact info.
- **Image Generate and Post** – User generates AI images and posts them on the platform.
- **View Request and Verify** – User checks commission requests and confirms them.
- **Chat with Artist** – User communicates directly with artists.
- **View Artist and Send Request** – User browses artist profiles and sends work requests.
- **Payment** – User makes secure payments for artwork or services.

- **Send Complaint and View Reply** – User submits complaints and checks admin replies.
- **Send Rating** – User gives ratings and feedback for artists.
- **View Work and Send Request** – User views artist works and sends commission requests

### 3.9.3.Artist

This module enables users to connect with professional artists. Users can send customization requests, and artists can refine, enhance, or recreate artworks. It includes features like chat, request verification, and project approval.

- **Register** – Artist creates an account on the platform.
- **Login** – Artist logs into the system securely.
- **Update Profile** – Artist updates bio, skills, and portfolio details.
- **View Posts and Send Requests** – Artist views user posts and sends proposals.
- **View Request Status** – Artist checks whether requests are approved or pending.
- **Chat with User** – Artist communicates with users regarding projects.
- **Manage Works** – Artist uploads, edits, or deletes their artworks.
- **Manage Skilled Area** – Artist updates their expertise categories (e.g., digital art, logo design).
- **View Request from User and Verify** – Artist reviews user requests and accepts/rejects them.
- **View Payment Status** – Artist checks payment confirmation details.
- **View My Rating** – Artist sees ratings and reviews received.
- **View Work Request and Verify** – Artist confirms commission requests.

## 3.10.PROJECT SHEDULING

For the successful completion of every project there must be detailed scheduling. The software development has different participating steps. First of all, We have done the requirement analysis phase. For this visit different sites, make an interaction with students for identifying different problems they are faced during their career selection. Then, We discuss these collected requirements with our project guide. After collecting the requirements, a detailed study of preliminary investigation is done. It includes six major questions.

1. What is being done?
2. How it is being done?
3. Done a problem exist?
4. If a problem exists how severe it is?
5. How frequently does it occur?
6. What is main reason for that problem?

| SL NO | PHASES | START DATE | END DATE | NO. OF DAYS |
|---|---|---|---|---|
| 1 | STUDY PHASE | 13-07-2025 | 20-07-2025 | 7 days |
| 2 | DESIGN PHASE | 24-08-2025 | 06-09-2025 | 13 days |
| 3 | DEVELOPMENT PHASE | 05-10-2025 | 25-11-202 | 20 days |
| 4 | TESTING | 20-12-2025 | 25-12-2025 | 21 days |
| 5 | IMPLEMENTATION | 04-12-2025 | 10-12-2025 | 7 days |

**TOTAL NO OF DAYS: 68**

## 3.11.SPRINT PLANNING AND MANAGEMENT

Sprint planning is a crucial part of Agile development, where the project is divided into smaller, manageable iterations known as Sprints Each Sprint delivers a functional part of the system and focuses on implementing and testing specific modules

The PIXEL PACT project was developed using an iterative Agile approach to ensure continuous improvement and timely delivery of features

### 3.11.1.SPRINT OVERVIEW

Each Sprint lasted for two weeks and focused on achieving specific goals related to module completion, testing, and user feedback.

The iterative cycle included Sprint planning, development, review, and retrospective.

### 3.11.2.SPRINT OBJECTIVES

- To break down the entire project into smaller deliverables.
- To achieve incremental progress in design, development, and testing.
- To integrate user and guide feedback continuously.
- To ensure each module is tested and validated before the next Sprint.

### 3.11.3.SPRINT DETAILS

| Sprint No | Duration | Key Tasks | Deliverables |
|---|---|---|---|
| Sprint 1 | 1st Week – 2nd Week | Requirements analysis, UI design, architecture planning | SRS, UI mockups, system design |
| Sprint 2 | 3rd Week – 5th Week | User authentication, role management, dashboard | Login system, user & artist modules |
| Sprint 3 | 6th Week – 8th Week | AI artwork generation, artwork upload, preview | AI art module, artwork management |
| Sprint 4 | 9th Week – 11th Week | Artist collaboration, blockchain ownership, security | Artist workflow, blockchain records |
| Sprint 5 | 12th Week –13th Week | Testing, optimization, documentation | Final system, test report, project report |

### 3.11.4.SPRINT REVIEW

At the end of each Sprint, a review meeting was conducted to assess progress, identify challenges, and plan improvements for the next Sprint.

This iterative feedback process ensured consistent quality and system enhancement.

### 3.11.5.SPRINT RETROSPECTIVE

The team analyzed each Sprint’s performance to learn from mistakes and enhance collaboration, communication, and efficiency in future cycles.

## 3.12.USER STORIES

| User Story ID No. | As a (type of user) | I want to (perform some Goal) | So that I can (achieve some goal) |
|---|---|---|---|
| 1. | Admin | Login | Manage and monitor the platform securely |
| 2. | Artist, User | Registration | Access the platform and its services |
| 3. | Admin | View User | Keep track of all registered users |
| 4. | Admin | Verify Artist | Ensure only genuine artists are approved |
| 5. | User | Generate image | Get customized artwork using AI |
| 6. | Artist | View post & send request | Find opportunities to work with users |
| 7. | User | View request & verify | Approve or reject requests before processing |
| 8. | Artist, User | Chat | Communicate directly between user and artist |
| 9. | Artist | Manage skills | Keep my profile updated with latest skills |
| 10. | Artist | Manage Work | Organize and showcase my completed works |
| 11. | User | View work & send request | Choose artwork and request customization |
| 12. | Artist | View request & update | Respond to user requests efficiently |
| 13. | User | View status &review | Track progress and share feedback |
| 14. | Artist | Payment | User Pay securely for the artwork/service to artist |
| 15. | Admin | View Rating | Monitor performance and quality of artist |
| 16. | User | Send complaint | Report issues and get support |
| 17. | Admin | View complaint and send replay | Resolve issues and maintain user trust |
| 18. | User | View reply | See admin's response to my complaint |
| 19. | Admin | Block/unblock Artist | Control platform misuse and maintain quality |

# 4
# SYSTEM REQUIREMENT SPECIFICATION

## 4.1.INTRODUCTION

System Analysis is a crucial phase in the software development life cycle (SDLC) that focuses on understanding, examining, and defining the functional and non-functional requirements of a system. In this phase, the existing system (if any) is studied to identify its limitations, inefficiencies, and challenges, followed by designing a new system that effectively addresses those issues. For PixelPact, system analysis involves a detailed examination of how digital art platforms currently operate, the problems faced by users and artists—such as lack of transparency, insecure transactions, and limited communication—and how a blockchain-based solution can overcome these barriers. This stage helps to clearly define user roles, system modules, data flow, and the interaction between components to ensure the final design aligns with real-world needs. Through careful analysis, PixelPact aims to build a robust, secure, and user-friendly platform that enhances trust, creativity, and collaboration between digital artists and clients.

## 4.2.SOFTWARE SPECIFICATION

Once the system requirements are identified, it is essential to select appropriate software tools that can effectively meet those requirements. This section describes the software components used in the development of the system.

| Software Component | Description / Purpose |
|---|---|
| Operating System | Windows 10 or above (for development and testing) |
| Backend Framework | **Node.js with Express.js (MERN Stack)** – for handling server-side logic, RESTful APIs, and business rules |
| AI Module | **Python (FastAPI )** – for AI-based image analysis, recommendations, and intelligent processing |
| Frontend Technologies | **React.js, HTML5, CSS3, Bootstrap, JavaScript** – for building a responsive and interactive web interface |
| Mobile Application Framework | **Flutter (Dart)** – for developing cross-platform mobile applications (Android & iOS) |
| Database Management System | **MongoDB** – for scalable, flexible, and efficient data storage |
| Integrated Development Environments (IDE) | PyCharm, Android Studio, or Visual Studio Code – for development and debugging |

| | |
|---|---|
| Web Server | **Node.js Server / Express.js** and **FastAPI Development Server** |
| API Communication | RESTful APIs using JSON |
| Version Control | Git and GitHub – for source code management and team collaboration |
| Blockchain Integration Tools | Blockchain SDKs and APIs (for implementing smart contracts ) |
| Browser Support | Google Chrome, Mozilla Firefox, or Microsoft Edge |

## 4.3.HARDWARE SPECIFICATION

The PixelPact system requires reliable yet cost-effective hardware to support development, testing, and deployment of web, mobile, AI, and blockchain modules. The hardware requirements are outlined below.

| Hardware Component | Minimum Requirement | Recommended Requirement |
|---|---|---|
| **Processor** | Intel Core i3 or equivalent | Intel Core i5 or above |
| **RAM** | 4 GB | 8 GB or above |
| **Storage** | 256 GB HDD | 512 GB or 1 TB **SSD (preferred)** |
| **Monitor** | 14-inch display | Full HD (1920×1080) display or higher |
| **Input Devices** | Standard Keyboard and Mouse | Wireless Keyboard and Mouse |
| **Internet Connection** | Stable broadband connection | High-speed broadband or fiber |
| **Mobile Device (for testing)** | Android 9.0+ or iOS 12+ | Latest Android/iOS device for optimal testing |

## 4.4.TECHNOLOGY SPECIFICATION

The PixelPact platform integrates modern and scalable technologies to ensure high performance, security, and an enhanced user experience. The following technologies are used in the development of the system:

| Technology | Purpose / Functionality |
|---|---|
| **Block chain Technology** | Ensures secure, transparent, and immutable records for artwork authenticity, ownership verification, and prevention of duplicate or fake submissions. |
| **Node.js & Express.js (MERN Stack)** | Handles server-side logic, user authentication, business rules, and RESTful API development. |
| **Python (AI Module – FastAPI)** | Manages AI-based image analysis, recommendation systems, and intelligent artwork generation features. |
| **Flutter (Dart)** | Enables the development of cross-platform mobile applications using a single codebase. |
| **MongoDB Database** | Provides flexible, scalable, and efficient NoSQL data storage for user profiles, artworks, and transaction metadata. |
| **React.js, HTML5, CSS3, Bootstrap, JavaScript** | Used for developing a responsive, interactive, and user-friendly web interface. |
| **RESTful APIs** | Facilitates seamless communication between web frontend, backend server, AI module, and mobile applications.. |
| **AI Image Generation** | Allows users to generate or enhance artwork concepts using integrated AI tools. |
| **Smart Contracts (Block chain – Non-payment)** | Stores artwork hashes and ownership proofs to verify originality and prevent unauthorized reuse of artworks. |
| **Git/GitHub** | Used for version control, source code management, and team collaboration during development. |

# 5

# DESIGN PHASE

## 5.1.INTRODUCTION

A software design is a meaningful engineering representation of some software product that is to be built. A design can be traced to the customer's requirements and can be assessed for quality against predefined criteria.

During the design process the software requirements model is transformed into design models that describe the details of the data structures, system architecture, interface, and components. Each design product is reviewed for quality before moving to the next phase of software development

Design forms a blue print of the system and adds how the components relate to each other. The design phase proceeds accordingly to an ordinary sequence of steps, beginning with review and assignment of task and ending with package design. Design phase is the life cycle phase in which the detailed design of the system selected in the study phase is accomplished. A smooth transition from the study phase to design is necessary because the design phase continues the activities in the earlier phase. The first step in the design phase is to design the database and then input and output within predefined guidelines.

## 5.2.ARCHITECTURAL DESIGN

Architecture is an overall structure of a system. Architecture takes into consideration the overall working of the system. Large system can be decomposed into sub-systems that provide some related set of services. The initial design process of identifying these sub- systems and establishing a framework for sub- system control and communication is called architecture design.

Architecture design usually comes before detailed system specification. Architecture decomposition is necessary to structure and organize the specification. There is no generally accepted process depends on application knowledge and on the skill and intuition of the system architect.

# 5.3.DATA FLOW DIAGRAM

Data Flow Diagram is the graphical description of the system's data and how the processes transform the data. Data Flow diagram depicts information flow, and the transform that are applied as data move from the input to output. It is the starting point of the design phase that functionally decomposes the requirement specifications down to the lowest level of details

Thus a DFD describes what data flows (logical) rather than how they are processed Unlike detailed flowchart, Data Flow Diagrams do no supply detailed description of the modules but graphically describes a system's data and how the data interacts with the system. A DFD consists of a series of bubble joined bylines. The bubble represents data transformation and lines represent data flow in the system. So DFD is also called Bubble Chart

**BASIC DFD SYMBOLS:**

To Construct a Data Flow Diagram with the help of following symbols.

- Arrow
- Circles
- Open End Box
- Squares

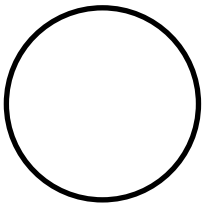
## ARROW

An arrow identifies the data flow in motion. It is a pipeline through which information is flown like the rectangle in the Flow Chart.

Data may flow a source to a processor and from a data store or process. An arrow line depicts the flow, with the arrowhead pointing in the direction of flow.

## CIRCLE

Circle stands for process that converts the data into information



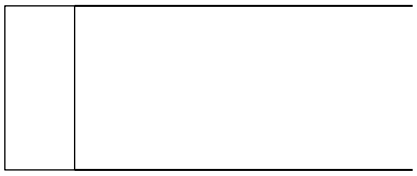
A process represents transformation where incoming data flow is changed into outgoing flows.

## RECTANGLE

A Rectangle defines a source or destination of system data. A source is a person or a part of organization, which enters or receives information from the system but is considered to be outside the context of the data flow model.

## OPEN END BOX

An Open End Box represents a data store, data at rest or temporary reposition of data



A graphical picture of the logical steps and sequence involved in a procedure or a program is called a flow chart. Unlike detailed flow chart, Data Flow Diagram does not supply detailed description of the modules but graphically describes a system’s data how the data interact with the system.

## SIX RULES FOR CONSIDERING DATA FLOW DIAGRAM

- Arrows should not cross each other
- Squares, circles and Data Store must have names
- Decomposed data flow squares and circles can have the same names.
- Choose meaningful names for data flow
- Draw all data flows around the outside of the diagram.
- Control information such as record count, password and validation Requirement is not relevant to Data Flow Diagram.

## 5.4.DIFFERENT LEVELS OF DATA FLOW DIAGRAMS:

A dataflow diagram as a diagram that depicts data source, data sinks, data storage and processes performed on data as nodes, and logical flow of data as links between the nodes. Generally DFD are used as a design notation to represent architectural design and top level design specification.

DFD represent the system in hierarchical manner with the one top level and many lower level diagrams with each representing separate parts of the system. Since diagrammatic representations are easier to interpret as compared to the technical descriptions, the non-technical users can also understand the system details clearly.

# LEVEL 0

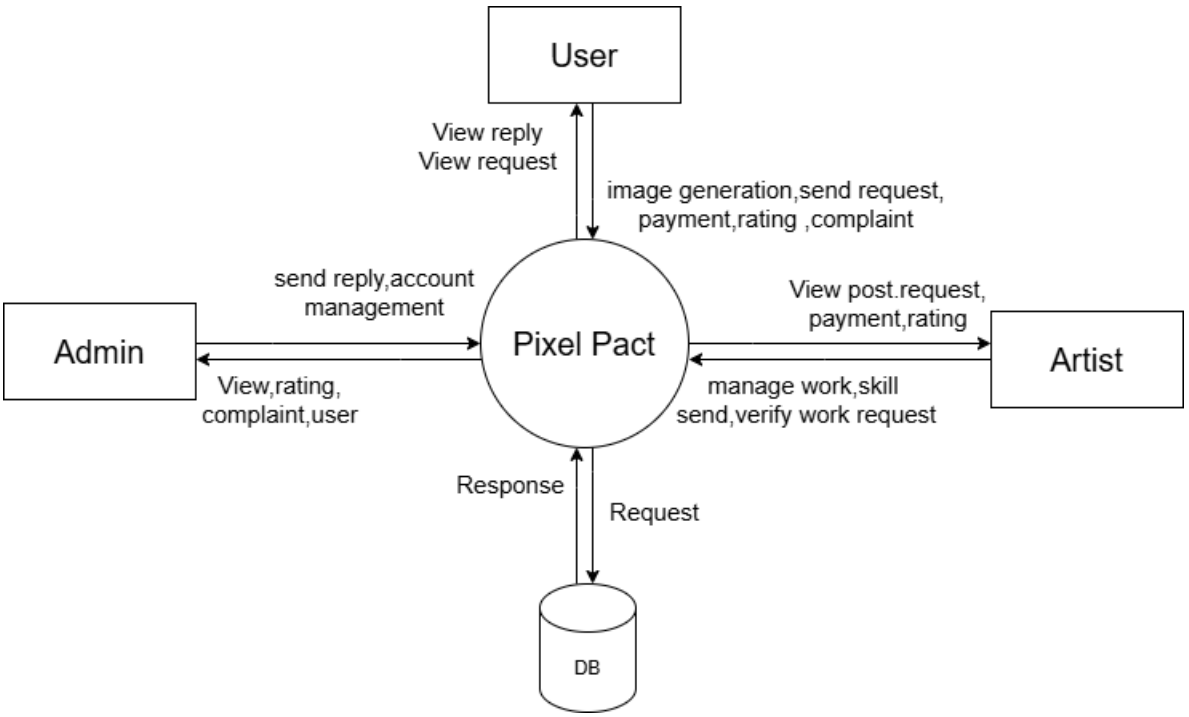



**Fig:** DFD of the Whole System

# LEVEL 1

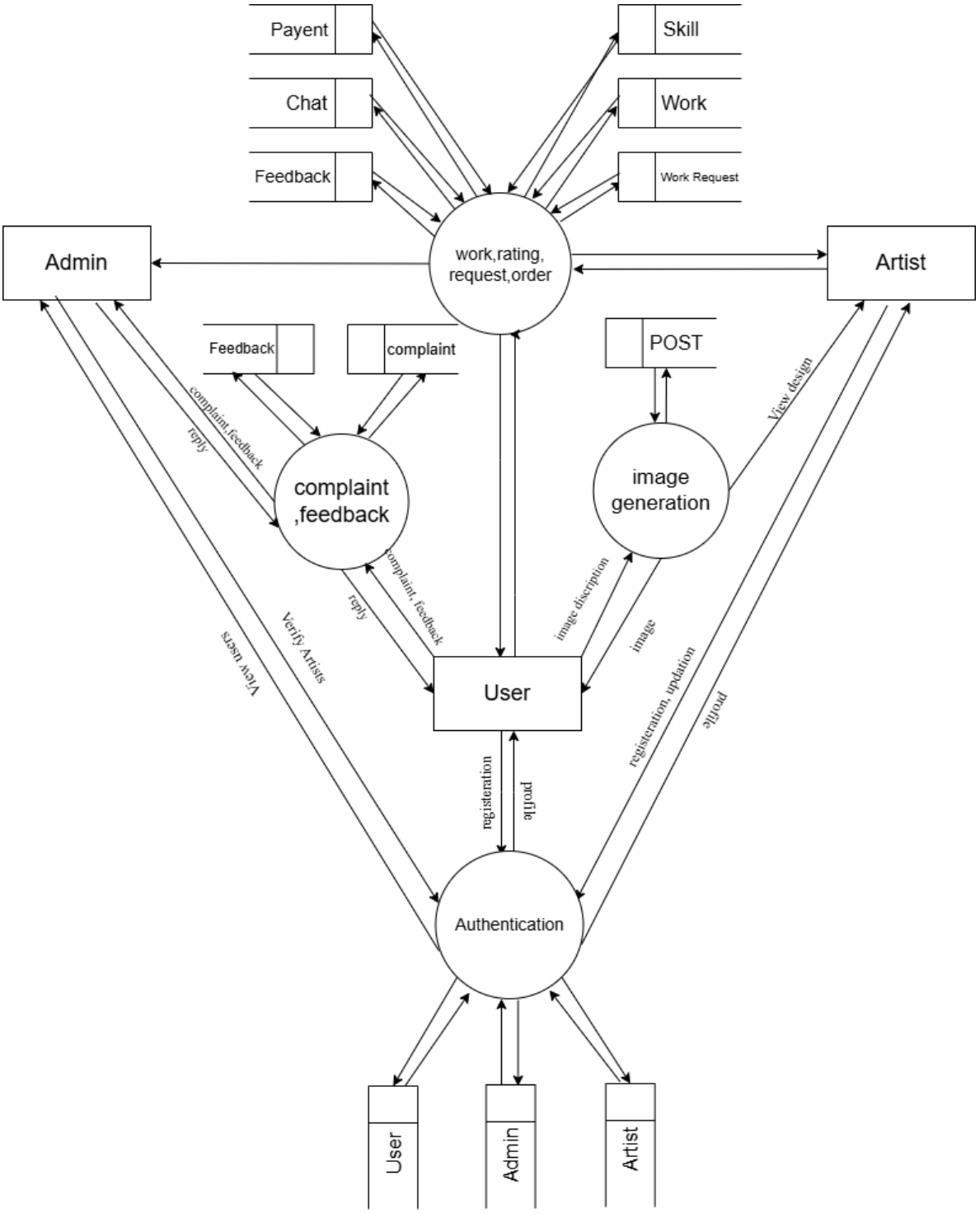

Payent
Skill
Chat
Work
Feedback
Work Request
Admin
work,rating, request,order
Artist
Feedback
complaint
POST
complaint,feedback
reply
View design
complaint ,feedback
image generation
complaint, feedback
reply
image discription
image
View users
Verify Artists
User
registeration, updation
profile
registeration
profile
Authentication
User
Admin
Artist

**Fig:** 4

LEVEL 1.1

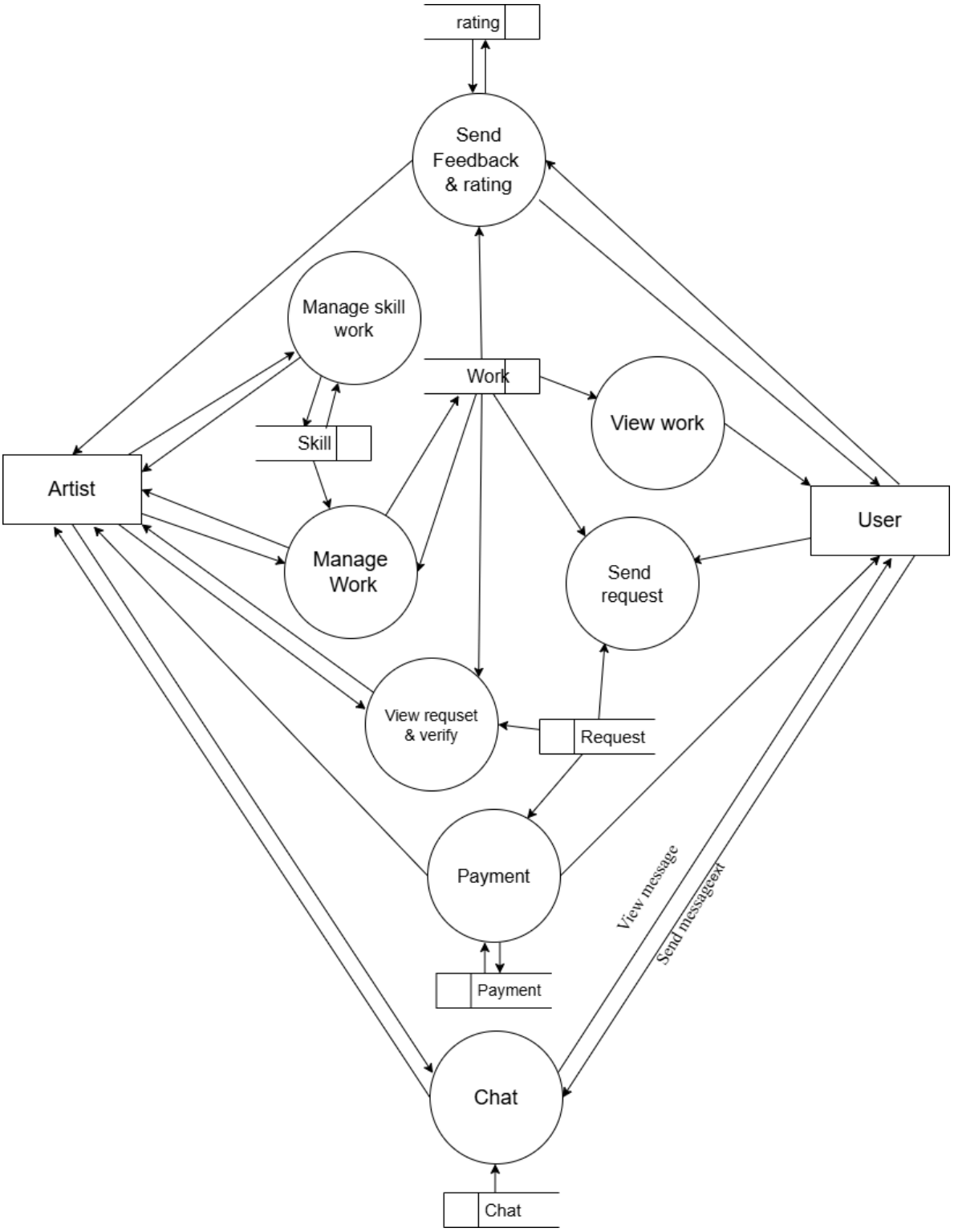
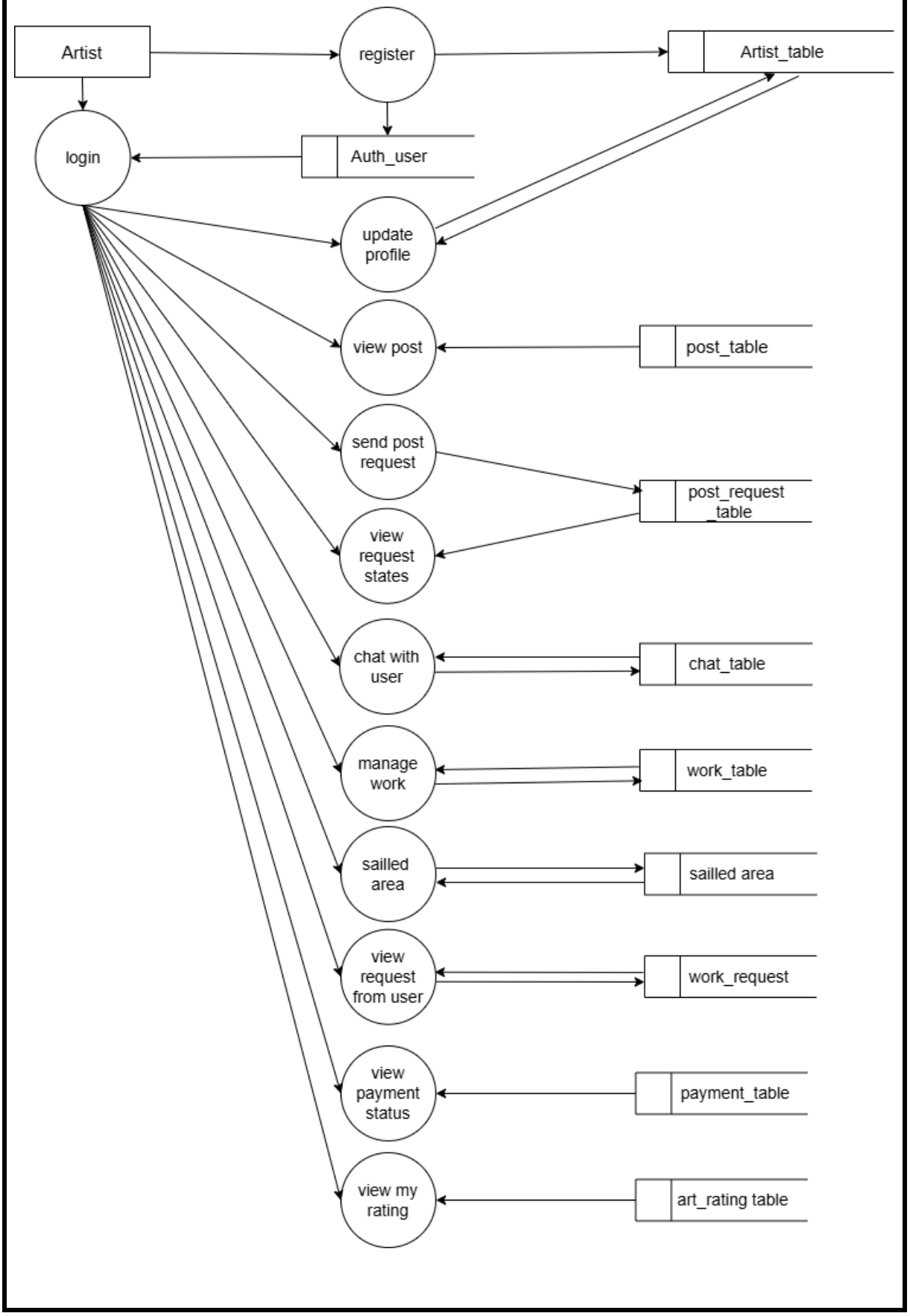

rating
Send Feedback & rating
Manage skill work
Work
View work
Skill
Artist
User
Manage Work
Send request
View requset & verify
Request
Payment
View message
Send messageext
Payment
Chat
Chat

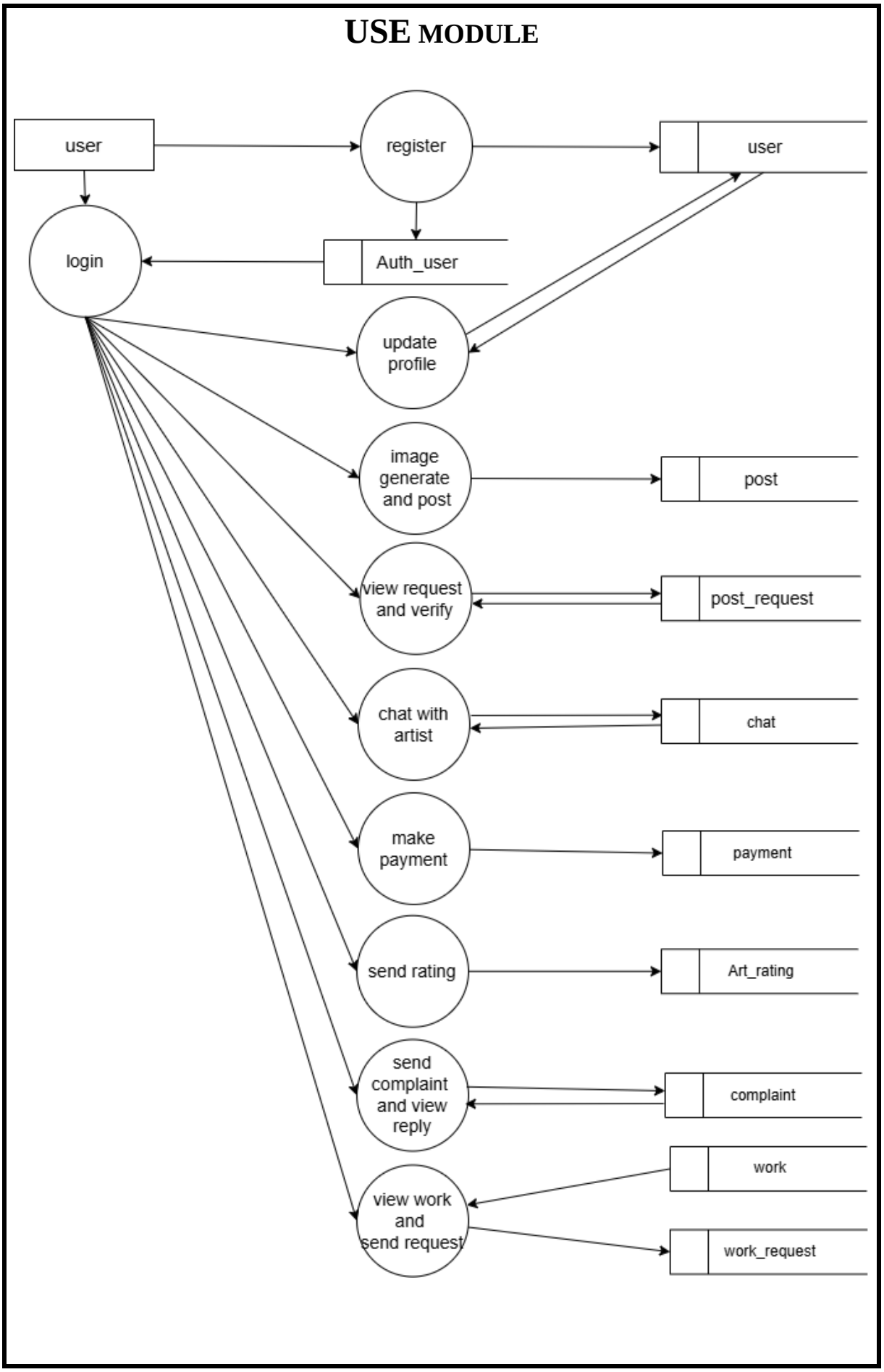
**Fig:**

# ADMIN MODULE

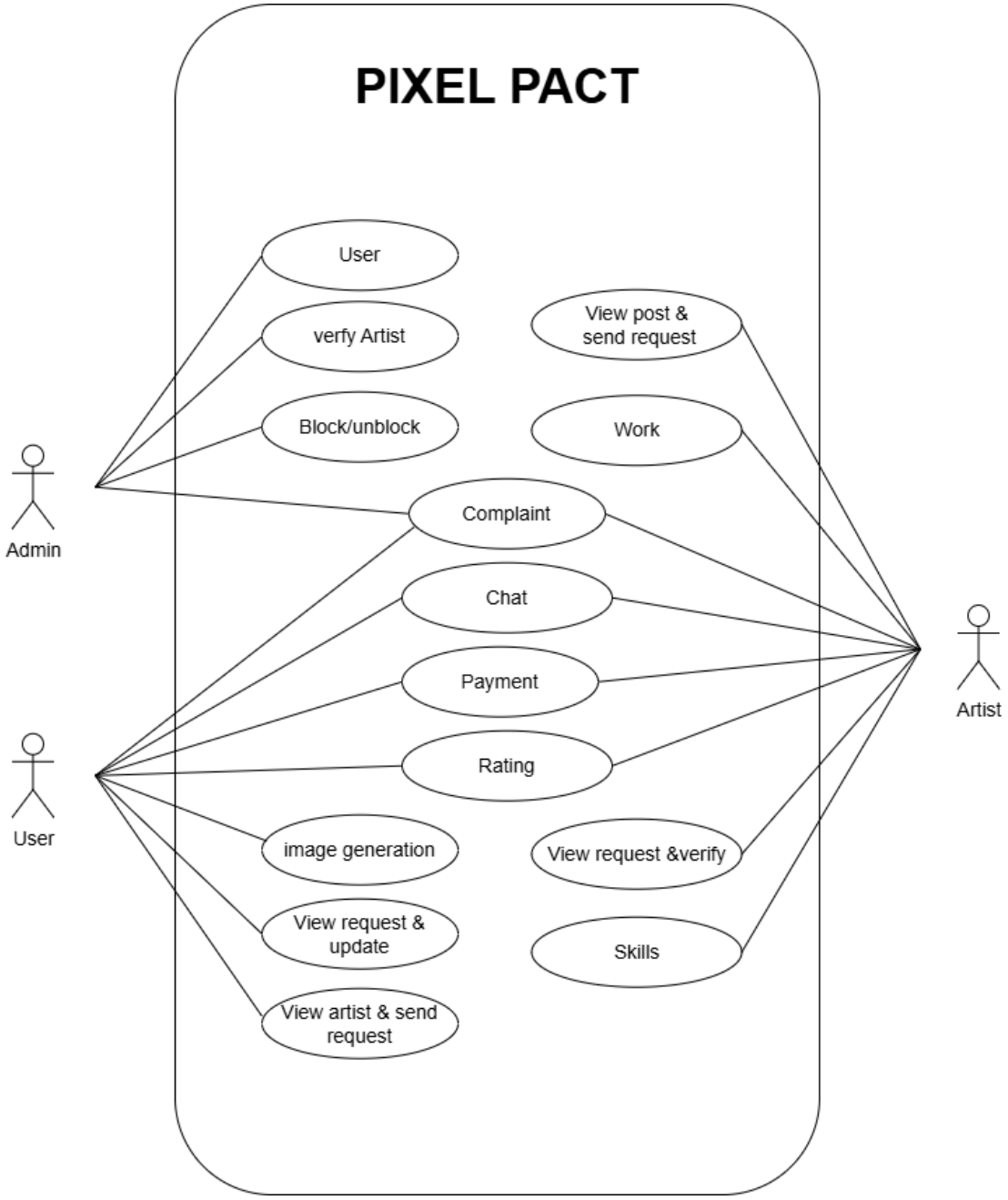
# ARTIST MODULE



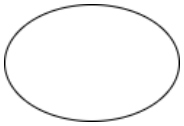
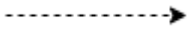
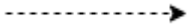
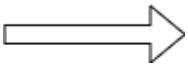
# USE MODULE


user
register
user
login
Auth_user
update profile
image generate and post
post
view request and verify
post_request
chat with artist
chat
make payment
payment
send rating
Art_rating
send complaint and view reply
complaint
work
view work and send request
work_request

# 5.5. USE CASE DIAGRAM



**UseCase symbols**

| Shape | Meaning / Usage |
|---|---|
| Stick Figure | Actor – represents a person or external system (User, Doctor, Admin, etc.) |
|  Oval | Use Case – represents a function or operation (e.g., “Log Glucose”) |
| ———— line | Association – connects actor to use case |
| «include»  (Dashed Arrow) | Include – mandatory relationship (e.g., “Place Order” includes “Make Payment”) |
| «extend»  (Dashed Arrow) | Extend – optional or conditional action (e.g., “Trigger SOS” extends “Log Glucose”) |
|  Hollow Arrow | Generalization – shows inheritance between actors or use cases |

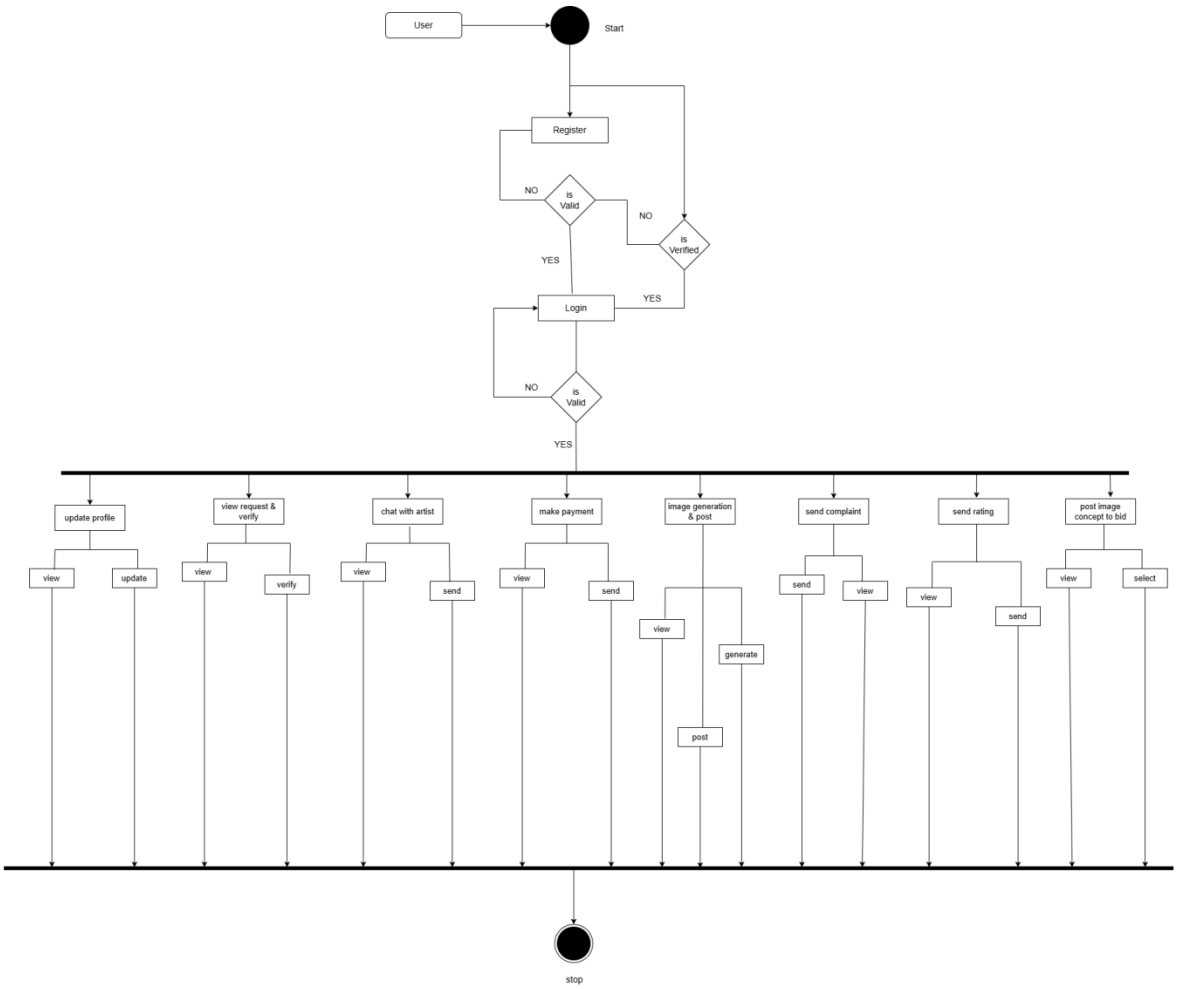
## 5.6. ACTIVITY DIAGRAM

The activity diagram illustrates the dynamic aspects of the system by showing the flow of control from one activity to another. It visually represents the sequence of activities involved in waste collection, user requests, pickup, and recycling processes

| Shape | Symbol Meaning | Use in Diagram |
|---|---|---|
| ● | **Start Node** | Beginning of the workflow. |
| ⊙ | **End Node** | Termination of the activity. |
| ⬭ | **Activity / Action State** | Represents a step or task in the process. |
| ◆ | **Decision Node** | Indicates a Yes/No branch in the flow. |
| → | **Control Flow Arrow** | Shows direction of execution between activities. |
| ‖ | **Swimlane** | Separates actors (User, Doctor, System, Pharmacy). |

| | | |
|---|---|---|
| ≡ | **Fork / Join Node** | Parallel process start or merge (if any in diagram). |

This diagram helps in understanding the workflow and overall process coordination within the system



**INPUT DESIGN**

Input design is the process of converting the user-originated inputs to a computer based format. The design for handling input specifies how data are accepted for computer processing. Input design is a part of overall system design that needs careful attention and if includes specifying the means by which actions are taken.

A system user interacting through a workstation must be able to tell the system whether to accept input produce a report system design or end processing. The collection of input data is considered to be the most expensive part of the.

Since the inputs have to be planned in such a manner so as to get the relevant information extreme care is taken to obtain the information. If the data going into the system is incorrect then processing and outputs will magnify these errors.

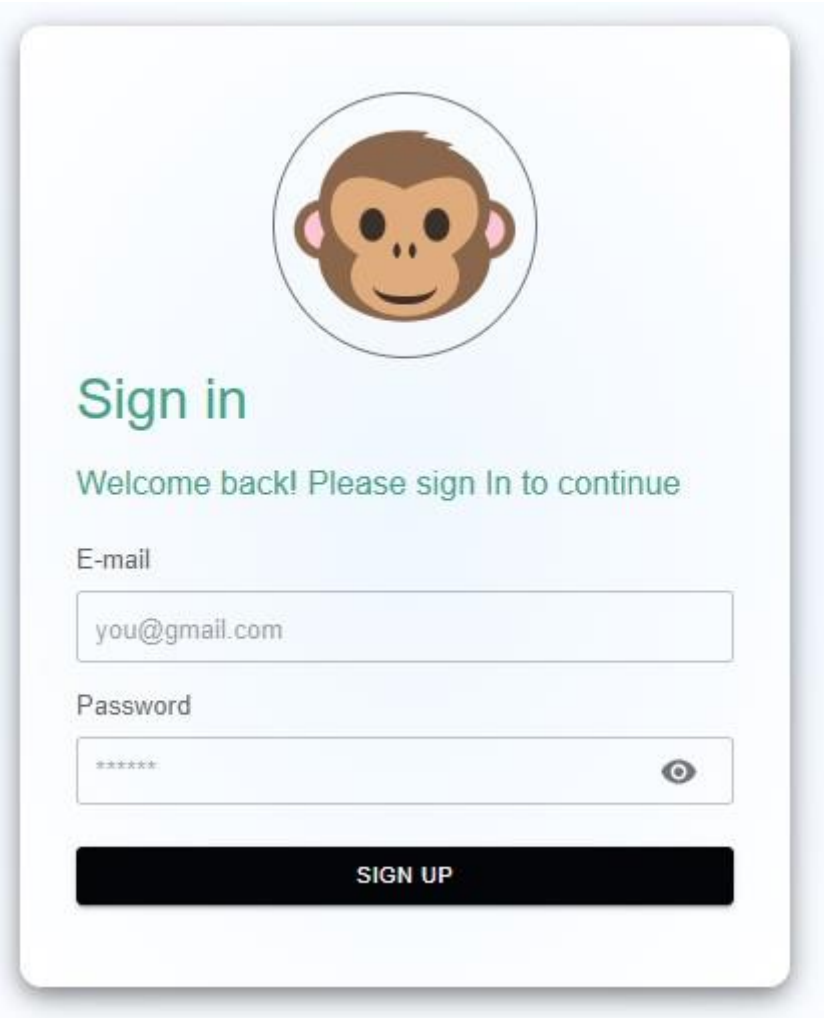
The major activities carried out are

- Collection of needed data from the source.
- Conversion of data into computer application form.
- Verification of converted data.
- Checking data for accuracy.

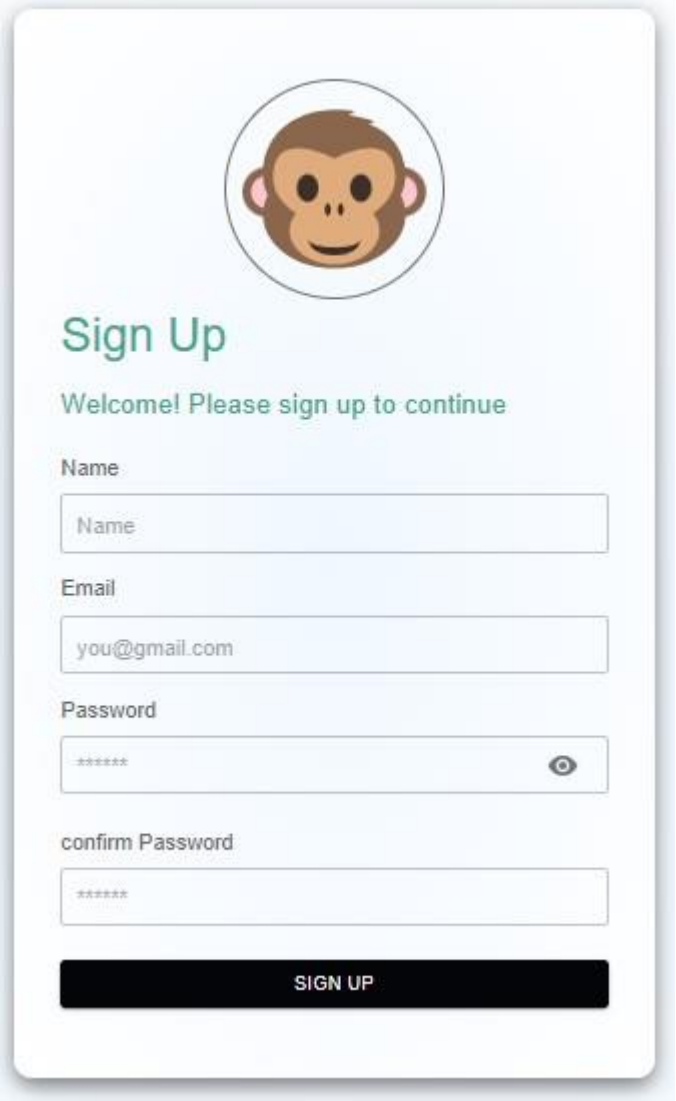
The goal of designing input data is to make the automation as easy and free from errors as possible. For providing a good input design for the application easy data input and selection features are adopted. The input design requirements such as user friendliness, consistent format and interactive dialogue for giving the right message and help the user at right time are also considered also considered for the development of this project.

**The following are the input of the system:**

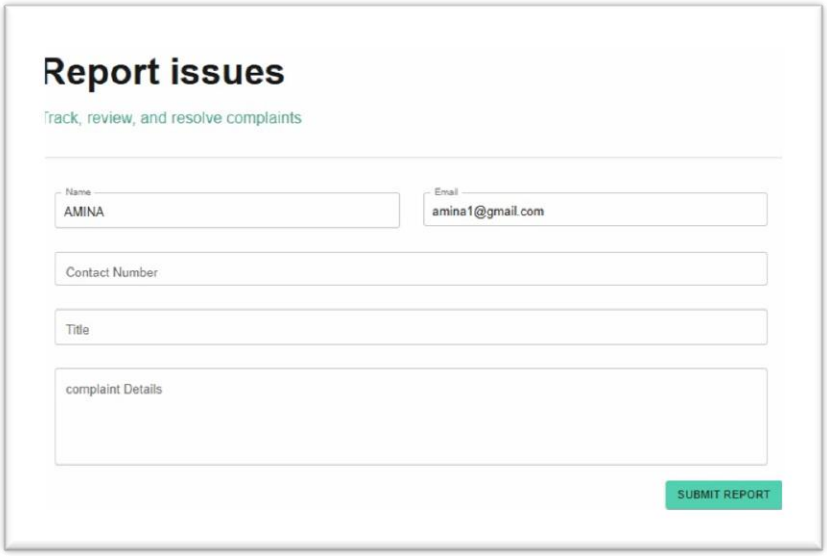
## LOGIN



## SIGN-IN

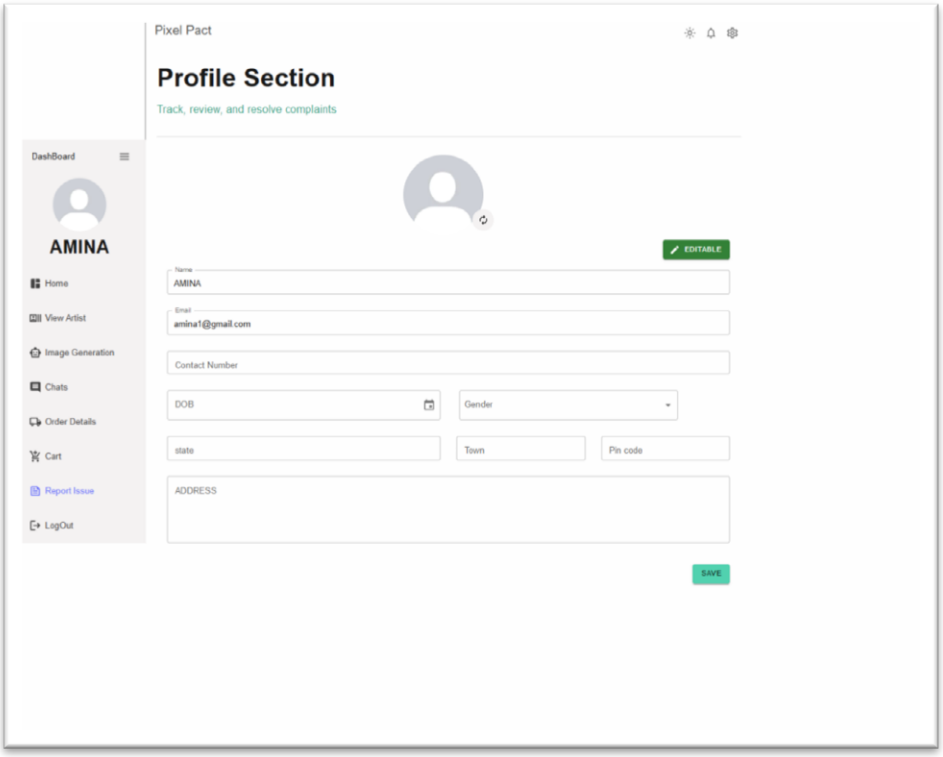



## COMPLAINT FORM

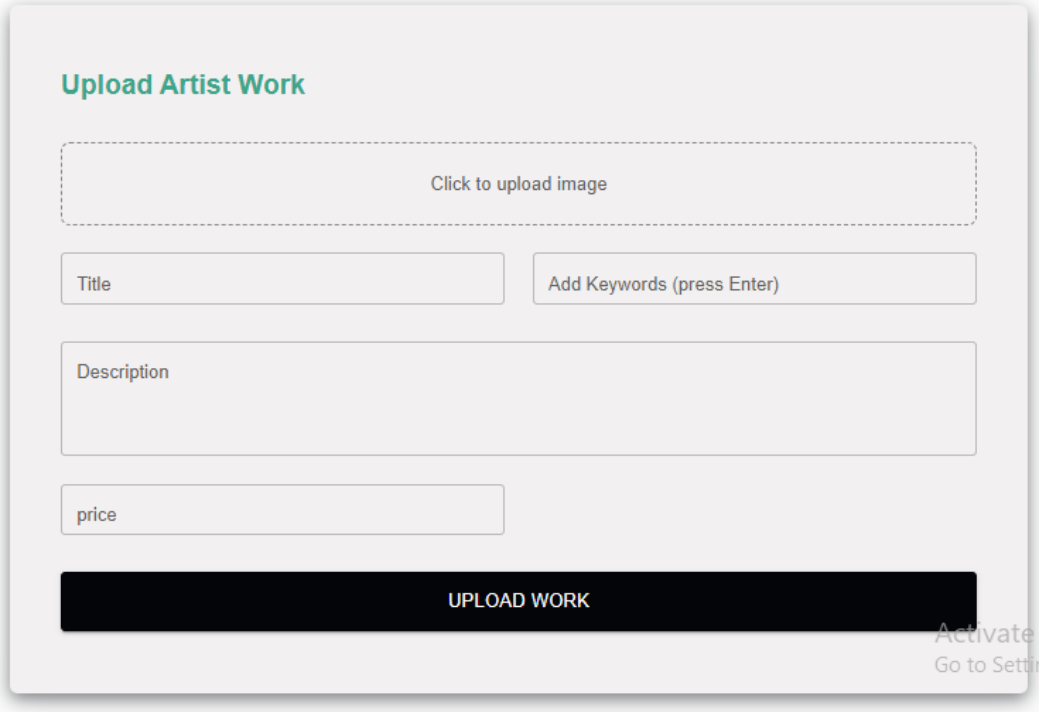


## ARTIST REGISTRATION

## PROFILE




## ADD WORK



## OUTPUT DESIGN

The output design is done so that the result of processing could be committed to the user and to provide a hard copy of these results and evaluations for later consultations. Effective output design will improve the clarity and performance of outputs. Output design phase of the system is concerned with the convergence of information's to the end user- friendly manner. The output design should be efficient, intelligible so that system relationship with the end user is improved and there by enhancing the process of decision making

.

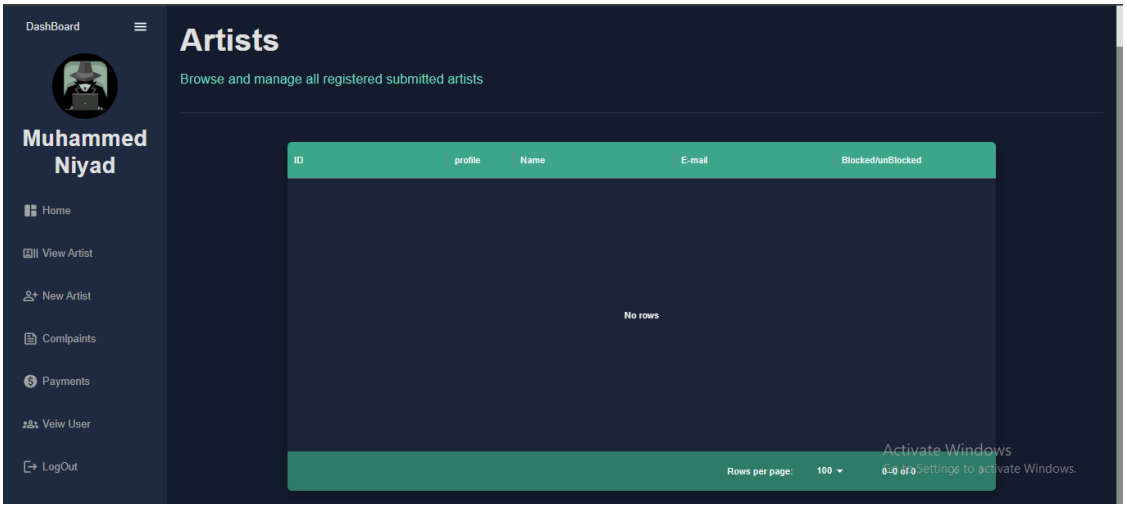
There are three types of outputs

- **External Output** - These outputs are prepared and given outside the organization. That is the modified database.
- **Internal Output** - These outputs are prepared exclusively for the people inside the organization Operational output.
- **Internal Output** - These outputs are prepared exclusively for the people inside the organization

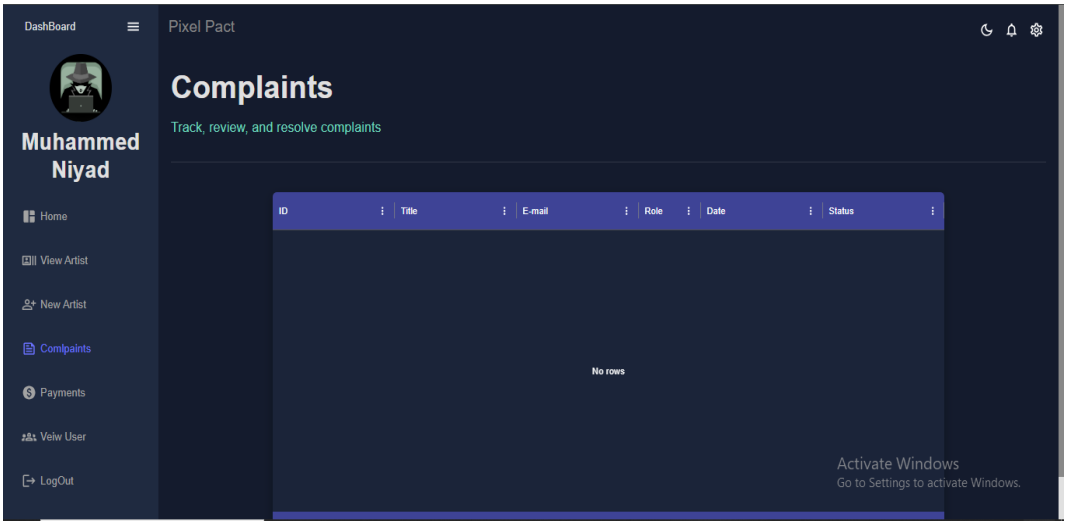
Output is the primary purpose of any system; therefore, output design is discussed before other aspects of the design.

The following are the major output screens of the system.

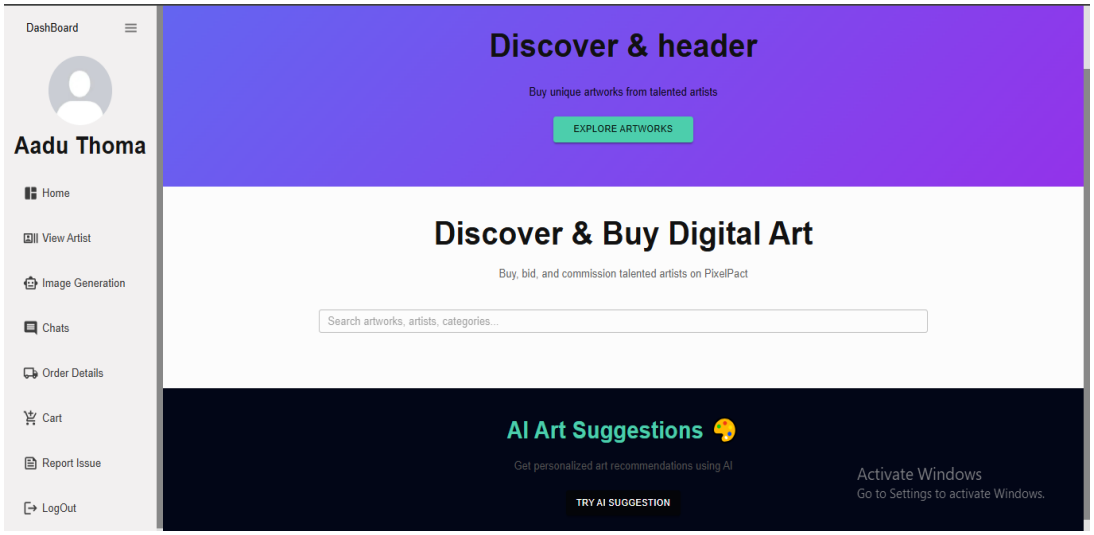
## ARTSIT LIST TABLE



## COMPLAINT LISTS




## USER DASBOARD




## USER CHATROOM

# DATABASE DESIGN (MongoDB)

Database design runs in parallel with application design. As system requirements are gathered, the data that needs to be stored, processed, retrieved, and presented is identified. For the PixelPact system, a flexible and scalable database is required to manage users, artworks, AI-generated content, artist collaborations, and blockchain ownership records efficiently.

PixelPact uses **MongoDB**, a NoSQL, document-oriented database management system. Instead of storing data in traditional tables and rows, MongoDB stores data in **collections and documents** using **JSON-like (BSON) format**, which allows dynamic schemas and easy scalability.

A well-designed database ensures efficient data storage, quick retrieval, high availability, and data security. MongoDB is suitable for PixelPact due to its ability to handle unstructured and semi-structured data such as images, prompts, metadata, and blockchain hashes.

## Objectives of Database Design

The main objectives of establishing the MongoDB database are:

- Reduce data redundancy where possible
- Support flexible and scalable data models
- Enable fast read and write operations
- Ensure data consistency and integrity
- Provide data independence from application logic
- Maintain privacy and security of user and artwork data

## Organizing the Data

In MongoDB, data is organized into **collections**, and each collection contains multiple **documents**. Each document represents a single record and can store complex nested data.

For the PixelPact system, the major collections include:

- Users
- Artists
- Artworks
- AI Generation Requests
- Collaborations
- Blockchain Records

Each document contains fields relevant to the entity and can embed related data where appropriate to improve performance.

## Identifying the Key

MongoDB automatically generates a unique **`_id`** field for every document, which acts as the **primary key**. This `_id` uniquely identifies each document within a collection.

Instead of traditional foreign keys, MongoDB uses **document references** or **embedded documents** to represent relationships between data. For example:

- An artwork document may store the user ID and artist ID
- Blockchain transaction details may be referenced inside the artwork document

This approach improves flexibility and reduces the need for complex joins.

## MongoDB Database Management System

MongoDB is an open-source, high-performance, NoSQL database management system. It supports:

- Schema-less document storage
- Horizontal scalability
- Indexing for fast queries
- High availability through replication
- Secure authentication and authorization

MongoDB is well-suited for modern web applications like PixelPact that require scalability, real-time performance, and handling of diverse data types.

## Data Consistency and Structure (Normalization in NoSQL Context)

Unlike relational databases, MongoDB does not strictly follow traditional normalization rules such as First Normal Form (1NF), Second Normal Form (2NF), and Third Normal Form (3NF). Instead, MongoDB follows a **denormalized data model** where related data can be embedded within a single document.

However, basic data structuring principles are followed:

- Avoid unnecessary duplication of large data
- Use embedded documents for closely related data
- Use references for frequently updated or shared data
- Maintain consistency at the application level

This approach improves performance and simplifies data retrieval while maintaining data integrity.

## Conclusion of Database Design

The MongoDB-based database design provides PixelPact with a flexible, scalable, and efficient data storage solution. By using document-oriented collections, embedded structures, and unique document identifiers, the system effectively manages complex relationships between users, artists, artworks, and blockchain records. This design supports future expansion, high performance, and secure data management for the PixelPact platform.

### 4.3.1. TABLE DESIGN & DESCRIPTION

**User Collection Structure (MongoDB)**

| Field Name | Data Type | Description |
|---|---|---|
| _id | ObjectId | Unique identifier for each user (auto-generated by MongoDB) |
| name | String | Name of the user |
| dob | Date | Date of birth |
| place | String | City or town of the user |
| post | String | Post office name |
| pin | String | Pin code |
| gender | String | Gender of the user |
| phone_no | String | User’s phone number |
| email | String | User’s email address (unique) |
| profile_image | String | Profile image or artwork image URL/path |
| role | String | Role of the user (User / Artist / Admin) |
| created_at | Date | Account creation date |

## Complaint Collection

**Description:** Stores complaints submitted by users against artists and the corresponding responses.

| Field Name | Data Type | Description |
|---|---|---|
| _id | ObjectId | Unique identifier for each complaint (auto-generated) |
| user_id | ObjectId | Reference to the user who submitted the complaint |
| artist_id | ObjectId | Reference to the artist against whom the complaint is made |
| complaint | String | Complaint details submitted by the user |
| reply | String | Response or resolution provided |
| status | String | Complaint status (Pending / Replied / Closed) |
| created_at | Date | Date and time when the complaint was submitted |
| updated_at | Date | Date and time of last update or reply |

## Artist Collection

**Description:** Stores artist profile details and contact information for the PixelPact platform.

| Field Name | Data Type | Description |
|---|---|---|
| _id | ObjectId | Unique identifier for each artist (auto-generated) |
| name | String | Name of the artist |
| dob | Date | Date of birth |
| place | String | City or town of the artist |
| post | String | Post office name |
| pin | String | Pin code |
| gender | String | Gender of the artist |
| phone_no | String | Artist’s phone number |
| email | String | Artist’s email address (unique) |
| profile_image | String | Profile image or artwork sample URL |
| bio | String | Short description or specialization |
| created_at | Date | Artist account creation date |
| status | String | Account status (Active / Inactive / Blocked) |

## Feedback Collection

**Description:** Stores feedback and ratings submitted by users for artists.

| Field Name | Data Type | Description |
|---|---|---|
| _id | ObjectId | Unique identifier for each feedback entry (auto-generated) |
| user_id | ObjectId | Reference to the user who submitted the feedback |
| artist_id | ObjectId | Reference to the artist receiving the feedback |
| feedback | String | Feedback or review details |
| rating | Number | Rating given by the user (e.g., 1.0 – 5.0) |
| created_at | Date | Date and time when the feedback was submitted |
| status | String | Feedback status (Visible / Hidden / Moderated) |

## Work Collection

**Description:** Stores details of artworks or projects created and uploaded by artists on the PixelPact platform

| Field Name | Data Type | Description |
|---|---|---|
| _id | ObjectId | Unique identifier for each work record (auto-generated) |
| artist_id | ObjectId | Reference to the artist who created the work |
| title | String | Title or name of the artwork |
| amount | Number | Price or payment amount associated with the work |
| description | String | Detailed description of the work |
| skill_area | String | Skill category of the work |
| image_url | String | Artwork image or preview path |
| created_at | Date | Date and time when the work was uploaded |
| status | String | Work status (Available / Sold / In Progress) |

## Request Collection

**Description:** Stores requests made by users for artworks or services offered by artists.

| Field Name | Data Type | Description |
|---|---|---|
| _id | ObjectId | Unique identifier for each request (auto-generated) |
| user_id | ObjectId | Reference to the user who made the request |
| work_id | ObjectId | Reference to the requested artwork or work |
| status | String | Current status of the request (Pending / Accepted / Rejected / Completed) |
| created_at | Date | Date and time when the request was made |
| updated_at | Date | Date and time of last status update |

## Chat Collection

**Description:** Stores chat messages exchanged between users and artists on the PixelPact platform.

| Field Name | Data Type | Description |
|---|---|---|
| _id | ObjectId | Unique identifier for each chat message (auto-generated) |
| from_id | ObjectId | Reference to the sender (User or Artist) |
| to_id | ObjectId | Reference to the receiver (User or Artist) |
| message | String | Content of the chat message |
| status | String | Message status (Sent / Delivered / Read) |
| created_at | Date | Date and time when the message was sent |

## Payment Collection

**Description:** Stores payment details related to user requests for artworks or services.

| Field Name | Data Type | Description |
|---|---|---|
| _id | ObjectId | Unique identifier for each payment (auto-generated) |
| request_id | ObjectId | Reference to the request for which the payment is made |
| amount | Number | Payment amount |
| status | String | Payment status (Pending / Completed / Failed / Refunded) |
| payment_method | String | Mode of payment (UPI / Card / Net Banking) |
| transaction_id | String | Unique transaction reference |
| created_at | Date | Date and time when the payment was made |

### Blockchain Collection

**Collection Name:** ImageBlock

**Description:** Stores blockchain records for artworks to ensure authenticity, integrity, and ownership

| Field Name | Data Type | Description |
|---|---|---|
| _id | ObjectId | Unique identifier for each blockchain block (auto-generated) |
| index | Number | Block index in the blockchain sequence |
| timestamp | Number | Time when the block was created (Unix timestamp) |
| data.artistId | ObjectId | Reference to the artist associated with the artwork |
| data.imageHash | String | Hash value of the artwork image |
| data.filename | String | Name of the uploaded image file |
| data.path | String | Storage path of the artwork image |
| data.uploadedAt | Date | Date and time when the artwork was uploaded |
| previousHash | String | Hash of the previous block in the blockchain |
| hash | String | Current block hash ensuring data integrity |

### Cart Collection

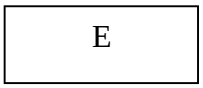
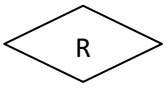
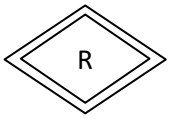
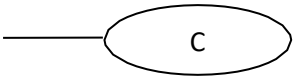
**Collection Name:** Cart

**Description:** Stores shopping cart details for users, including selected artworks, pricing, and cart status

| Field Name | Data Type | Description |
|---|---|---|
| _id | ObjectId | Unique identifier for the cart (auto-generated) |
| userId | ObjectId | Reference to the user who owns the cart |
| items | Array | List of cart items |
| totalPrice | Number | Total price of all items in the cart |
| status | String | Cart status (active / checked_out) |
| createdAt | Date | Date and time when the cart was created |
| updatedAt | Date | Date and time of last cart update |

## DIAGRAM DESCRIPTION

A model is a collection of conceptual tools for describing data, data relationship, data semantics, and consistency constrains. The E-R data model is a high-level conceptual data model that provides the designers with a collection of conceptual tool for describing the structure of a database. The E-R model perceives the real world as consisting of basic objects, called entities, and relationship among these objects. It developed to facilitate the database description by allowing specification of schema that represents the overall logical structure of a database. An E-R diagram can overall logical structure of a database graphically. E-R diagrams are simple and clear. An E-R diagram consists of the following major components

| SYMBOL | MEANING |
|---|---|
|  | Entity Set |
| | Weak Entity |
|  | Relationship Set |
|  | Identifying relationship for weak entity |
| | Attribute |
|  | Key attribute |
| | Multivalued attribute |

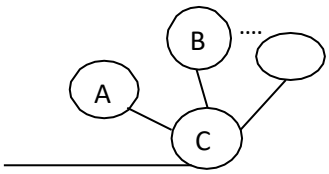
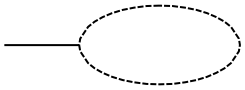
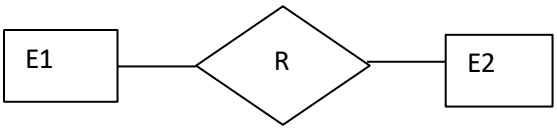
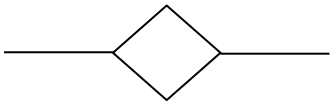
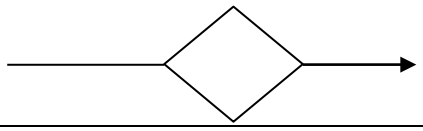
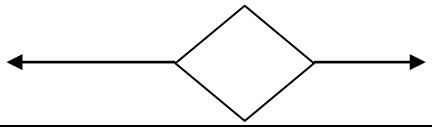
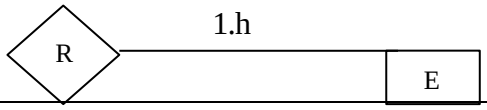
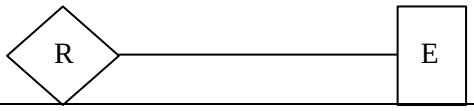
| | |
|---|---|
|  | Composite attribute |
|  | Derived attribute |
|  | Total participation of E2 in R |
|  | One-to-one relationship |
|  | Many-to-one relationship |
|  | Many-to-many relationship |
|  | Cardinality limit |
|  | Role Indicator |

**Table 4.4.1 ER Diagram And Description**

# ER DIAGRAM

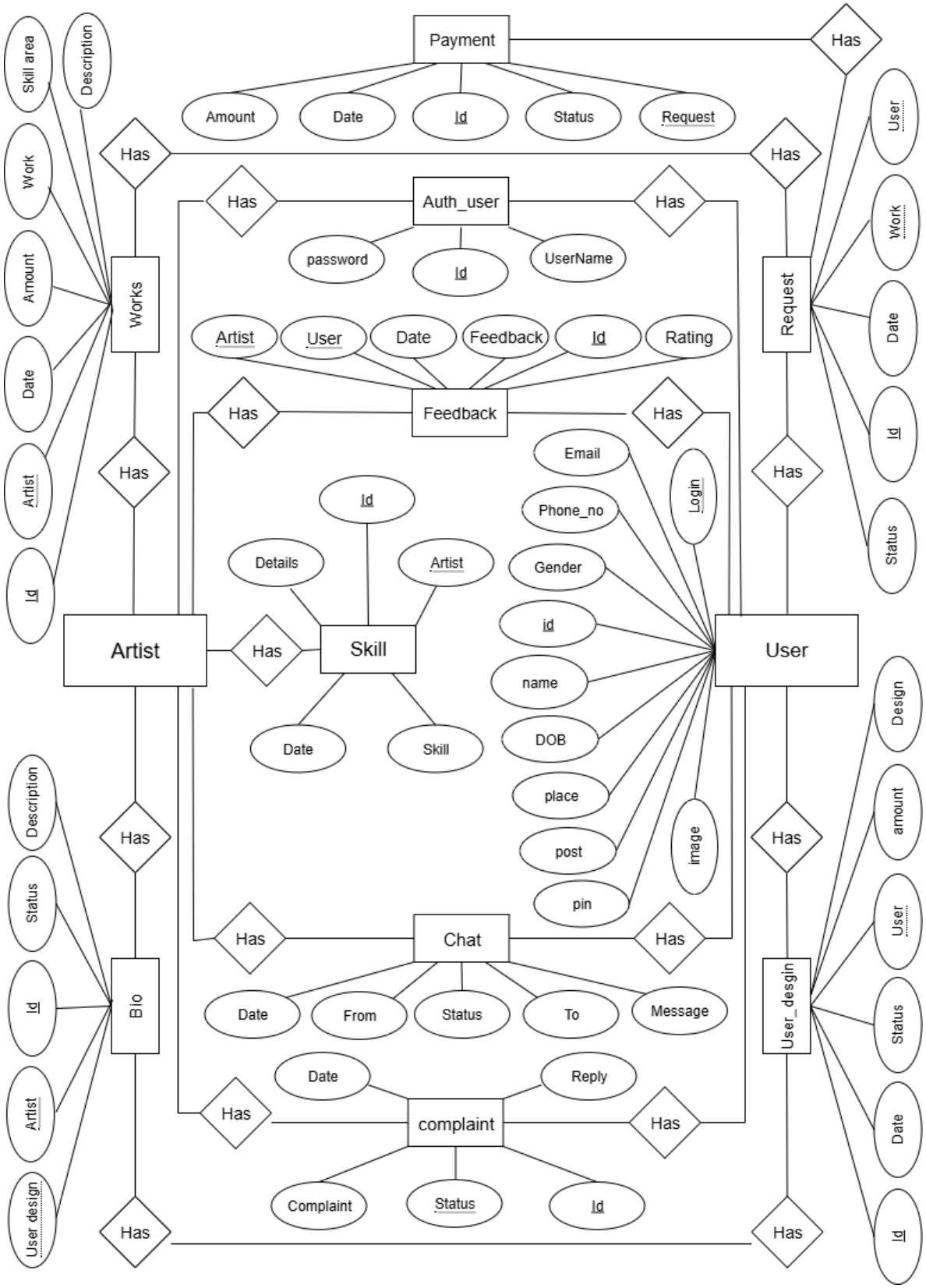



**Fig:**

# 6

# CODING

# CODING METHODOLOGY

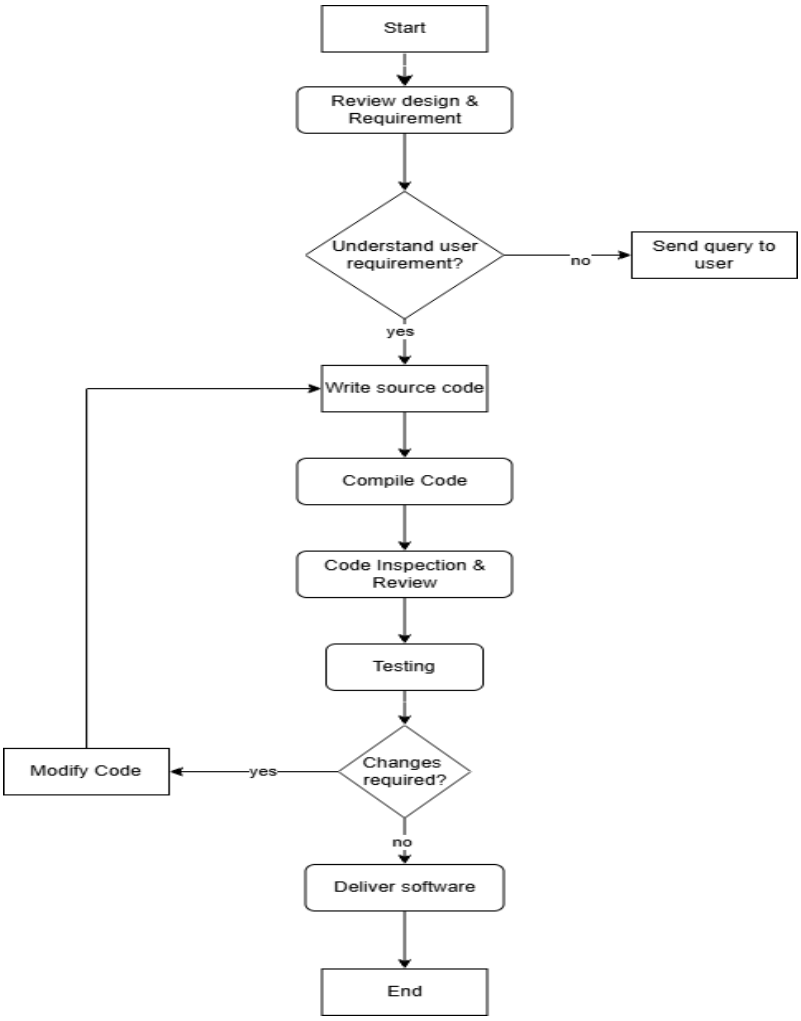
This methodology refers to a set of well-documented procedures and guidelines used in the analysis, design, and implementation of programs. Coding methodology includes a diagrammatic notation for documenting the results of the procedure. It also includes an objective set (ideally quantified) of criteria for determining whether the results of the procedure are of the desired quality. The steps to use coding' methodology are listed below.

1. The software development team begins its work by reviewing and understanding the design and requirements specification documents. These documents are essential for understanding user requirements and creating a framework for the software code.
2. In case the software development team is unable to understand user requirements correctly and further clarification is required, the queries are sent back to the user. In addition, the software development team also returns the requirements that are understood by them.
3. After the requirements are clearly understood by the software development team, the design and specifications are implemented in source code, supporting files, and the header files. Note that while writing the software code, the coding style guidelines should be followed. In some cases, there may be a proposal of change in hardware or software specifications. However, the requests for change are implemented only after the approval of the user.
4. When the software code is completely written, it is compiled along with otherrequired files.
5. Code inspection and reviews are conducted after the compilation. These methods are used to correct and verify errors in the software code.
6. Software testing is carried out to detect and correct errors in each module of the software code.
7. After the software code is tested, the software is delivered to the user along with the relevant code files, header files, and documentation files.

In Software Coding Process further change and clarifications are required in the design or SRS, the software development team raises a query, which is

sent to the user with the document containing what the software development team understood from the documents sent by the user. Changes are made only when the user has a positive response to the queries raised by the software development team.

## FLOWCHART OF SOFTWARE CODING



# CODING GUIDELINES

Writing an efficient software code requires a thorough knowledge of programming. This knowledge can be implemented by following a coding style which comprises several guidelines that help in writing the software code efficiently and with minimum errors. These guidelines, known as coding guidelines, are used to implement individual programming language constructs, comments, formatting, and so on. These guidelines, if followed, help in preventing errors, controlling the complexity of the program, and increasing the readability and understandability of the program.A set of comprehensive coding guidelines encompasses all aspects of code

development. To ensure that all developers work in a harmonized manner (the sourcecode should reflect a harmonized style as a single developer had written the entire code in one session), the developers should be aware of the coding guidelines before starting a software project. Moreover, coding guidelines should state how to deal with the existing code when the software incorporates it or when maintenance is performed.

Since there are numerous programming languages for writing software codes, each having different features and capabilities, coding style guidelines differ from one language to another. However, there are some basic guidelines which are followed in all programming languages. These include naming conventions, commenting conventions, and formatting conventions.

## NAMING CONVENTION

There are certain rules for naming variables, functions and methods in the software code. These naming conventions help software; developers in understanding the use of a particular variable or function. The guidelines used to assign a name to any variable, function, and method are listed below.

1. All the variables, functions, and methods should be assigned names that make the code more understandable to the reader. By using meaningful names, the code can be self- explanatory, thus, minimizing the effort of writing comments for variables. For example, if two variables are required to refer to 'total amount' and

'Artist name', they should be assigned names such as 'totalAmount' and 'artistName'.

2. For names, a full description in a commonly spoken language (for example, English) should be used. In addition, the use of abbreviations should be avoided. For example, variable names like 'contact Number' and 'address' should be used instead of 'cno' and 'add'.

3. Short and clear names should be assigned in place of long names. For example, 'multiply The Two Numbers' can be shortened to 'multiply Numbers' as it is clear and short enough to be expressed in reasonable length.

| Variable Naming Conventions | Constant Naming Conventions |
|---|---|
| Variable names should be written in **camelCase**, starting with a lowercase letter. For example, use totalAmount instead of Total Amount. | Constants should be written in **UPPERCASE letters**. If multiple words are used, they should be separated with an underscore.<br>Example: MAX_FILE_SIZE, BLOCKCHAIN_NETWORK_ID, |
| Temporary variables that are restricted to a small segment of code should have **short and meaningful names**. For example, temp can be used for temporary storage. Variables like i, j, and k are commonly used in loops. A single temporary variable should not be reused for different purposes within the same program. | The use of **literal values** should be avoided in the code. Directly using numbers like 15 may confuse the reader and lead to logical errors. Instead, meaningful constants should be defined. However, 0 and 1 can be used directly where appropriate. |
| The use of numbers in variable names should be avoided. For example, firstNumber should be used instead of number1. | Constants should represent **fixed values** that do not change during program execution, improving readability and maintainability. |

In every programming language, there is a different naming convention for variables and constants in the software code. The commonly used conventions for naming variables and constants are listed in Table.

As with variables and constants, there are some guidelines that should be followed while naming functions in the software code. These conventions are listed below.

- The names of functions should be meaningful and should describe the purpose of the function with clarity and briefness. Like variables, the names should be self-explanatory so that no additional description about the task of that function is required.
- The function name should begin with a verb function that displays the output on the screen. In case the verb itself is not descriptive, an additional noun or adjective can be used with the verb. For example, the function name 'add Marks' should be used to clarify the function and its purpose.
- In case the function returns a Boolean value, the helping verbs 'is' and 'has' should be used as prefixes for the function name. For example, the function name 'is Deposited' or 'has Deposited' should be used for functions that return true or false values

Comments are helpful in proper understanding of the code segment used in program. Commenting conventions should be used efficiently to make the code easy to grasp Generally, two types of commenting conventions are used: file header comments and trailing comments.

**File header comments** are useful in providing information related to a file as a whole and comprise identification information such as date of creation, Dame of the creator, and a brief description of the software code.

Trailing comments are used to provide explanation of a single line of code. These comments are used to clarify the complex code. These also specify the function of the abbreviated variable names that are not clear. In some languages, trailing comments are used with the help of a double slash (//).The commenting conventions

that are commonly followed in the software code are listed below.

- Comments should not be used to include information that is clearly understandable from the software.
- Comments should be used with important segments of code and code segments that are difficult to understand.
- Comments should be separated from the code to enhance readability of the software code.

Formatting (way of arranging a program in order to enhance readability) consists of indentation, alignment, and use of white spaces in the program.

Consistency plays an important role while formatting a program in an organized way. A program with consistent formatting makes the code easier to read and understand. The commonly used formatting conventions are listed below.

Indentation: This refers to one or more spaces left at the beginning of statements in the program. Indentation is useful in making the code easily readable. However, the spaces used for indentation should be followed in the entire program. The guideline that are commonly followed while indenting program are listed below.

White spaces: These improve readability by minimizing the compactness of the code. Some of the guidelines for proper usage of spaces within the code are listed below.

- Indentation should be used to highlight a nested block. Some nested blocks can be made with the help of 'if-else' and 'do-while' loops.
- Indentation is required if the statement is large enough to fit in a single line.
- Indentation should b consistent at the beginning and at the end of the braces in the program.
- There should be a space after placing a comma between two function arguments.
- There should be no space between a function name and parenthesis.
- There should be spaces to align the operators vertically to emphasize program structure and semantics.

## IMPLEMENTING CODING GUIDELINES

Implementing proper coding guidelines helps in identifying errors during the early stages of development. Early detection improves software performance, reduces maintenance cost, and ensures better readability and scalability of the system. In the PixelPact project, standardized coding practices are followed across frontend, backend, mobile application, and database layers to ensure consistency, security, and maintainability.

### Coding Guidelines Followed in PixelPact

### General Coding Guidelines

- All source code files are properly commented to explain the purpose of classes, functions, and critical logic.
- Proper indentation and formatting are maintained for better readability.
- Meaningful and descriptive variable, function, and class names are used.
- Unused variables and redundant code are avoided.
- Error handling is implemented using standardized error messages and HTTP status codes.
- Each file includes basic information such as author name, last modified date, and version (where applicable).
- Modular coding practices are followed to improve code reuse and maintenance.

#### Backend Coding Guidelines (Node.js / Express / Python)

- MVC (Model–View–Controller) architecture is followed for better separation of concerns.
- Each API endpoint is clearly defined and documented.
- Validation is performed for all incoming requests.
- Proper middleware is used for authentication, authorization, and error handling.
- All conditional statements use curly braces even if they contain a single statement.
- Errors follow a standard format: ErrorCode: ErrorDescription ,Example:
  `AUTH_401: Unauthorized access`

## Frontend Coding Guidelines (React / Flutter)

- Components are modular and reusable.
- State management is handled efficiently.
- UI components follow consistent naming conventions.
- Responsive design principles are implemented.
- Comments are added for complex UI logic.
- Assets such as images and icons are organized into appropriate folders.

## Database Coding Guidelines (MongoDB )

- Meaningful collection names are used.
- Every collection includes a unique identifier (ObjectId).
- Relationships between collections are maintained using references.
- Field names are written in camelCase for consistency.
- Required fields are properly validated at the schema level.
- Timestamps are enabled to track data creation and updates.

## Security and Performance Guidelines

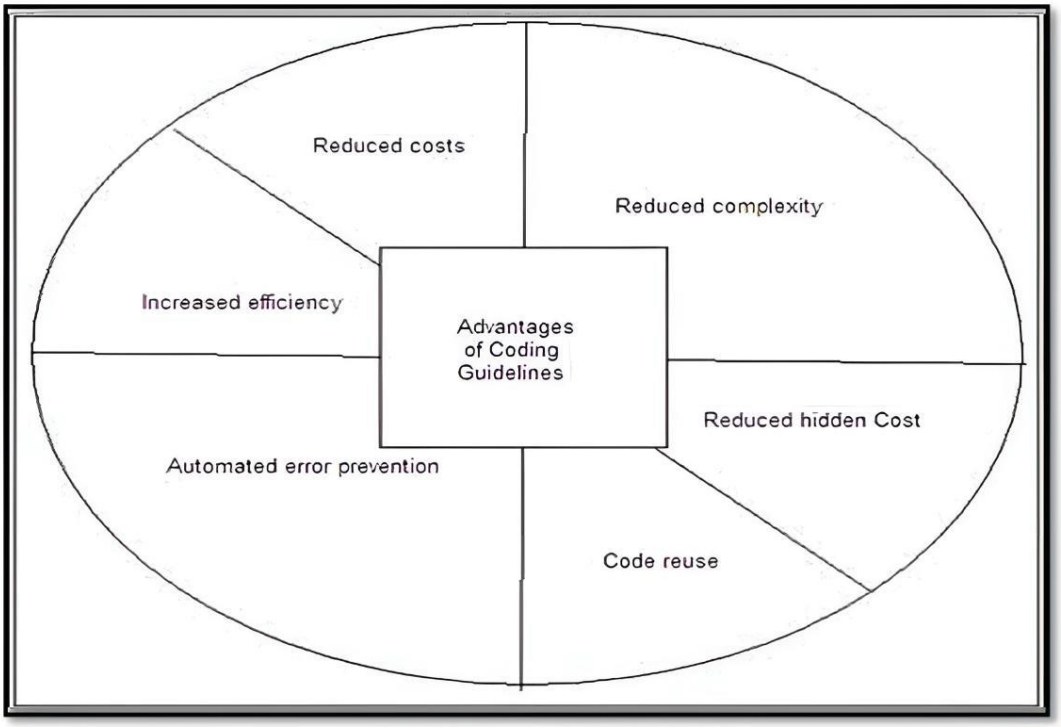
- Environment variables are used for sensitive data such as API keys and database credentials.
- Authentication tokens are securely managed.
- Input sanitization is applied to prevent injection attacks.
- Optimized queries are used to improve database performance.

By following these coding guidelines, the PixelPact system ensures high-quality code, improved performance, easier debugging, and long-term maintainability.

.

## ADVANTAGES OF CODING GUIDELINES

Coding guidelines supplement the language standard by defining acceptable and unacceptable usage of the programming language used. Acceptable usage avoids troublesome situations while unacceptable usage is conducive to errors or leads to misunderstanding of the written code. Properly implemented coding guidelines help the developer to limit program complexity, establish the basis for code review, and guard against compiler and common programming errors. Other advantages associated with coding guidelines are listed below and depicted.




- **Increased efficiency:** Coding guidelines can be used effectively to save time spent on gathering unnecessary details. These guidelines increase the efficiency of the software team while the software development phase is carried out. An efficient software code is fast and economical. Software coding guidelines are used to increase efficiency by making the team productive, thus, ensuring that the software is delivered to the user on time
- **Reduced costs:** Coding guidelines are beneficial in reducing the cost incurred on the software project. This is possible since coding guidelines help in detecting errors in the early stages of the software development. Note that if errors are discovered after the software is delivered to the user, the process of rectifying them becomes expensive as additional costs are incurred on late detection, rework, and

retesting of the entire software code.

- **Reduced hidden costs:** Coding guidelines, if adhered to in a proper manner, help to achieve a high-quality software code. The software quality determines the efficiency of the software. Software quality is the degree to which user requirements are accomplished in the software along with conformity to standards. Note that if quality is not considered while developing the software, the cost for activities such as fixing errors, redesigning the software, and providing technical support increases considerably

- **Code reuse:** Using coding guidelines, software developers are able to write a code that is more robust and create individual modules of the software code. The reason for making separate code segment is to enable reusability of the modules used in the software. A reusable module can be used a number of times in different modules in one or more software.

- **Automated error prevention:** The coding guidelines enable Automated Error Prevention (AEP).This assures that each time error occurs in software, the software development activity is improved to prevent similar errors in future. AEP begins with detecting errors in the software, isolating its cause, and then searching the cause of error generation. Coding guidelines are useful in preventing errors as they allow implementation of requirements that prevent the most common and damaging errors in the software code.

# 7

# TESTING PHASE

## INTRODUCTION

Software testing is critical element of software quality assurance and represent the ultimate review of the specification, design and coding. System testing makes a logical assumption that all the part of the system is correct; the goal will be successfully achieved.

Testing is a set of activity that can be planned in advance and conducted. Systematically, this is aimed at ensuring that the system works accurately and efficiently before live operations commences. Testing is the process of correcting a program with intend of finding an error. A good test case is one that has high probability of finding a yet undiscovered error. A successful test is one that uncovers a yet undiscovered error

## TESTING OBJECTIVES

There are several rules that can serve as testing objectives Testing is a process of executing a program with the intent of finding an error. A good test case is one that has high probability finding an undiscovered error. A successful test is one that uncovers an undiscovered error.

Testing is vital to the success of the system. System testing makes a logical assumption that if all parts of the system are subject to variety of tests on-line response, volume, stress, recovery and security and usability tests. A series of tests are performed before the system is ready for user acceptance testing.

## TESTING STRATEGIES

Testing strategies provide a road map for the software developer. The testing strategy must incorporate test planning, test case design, test execution, and resultant data collection and evaluation.

## WHITE BOX TESTING

White box testing is also known as code testing. The code testing strategy checks for the correctness of every statement in the program. To follow this strategy, there should be cases that result in execution of every instruction in the program or module, which is every path in the program, is tested. The test cases should be guaranteed that independent paths within module are executed once.

The test cases should be guaranteed that:

- Independent paths within a module are executed at least once
- All logical decisions are exercised on their true or false sides
- All loops are executed at their boundaries and within their operational bounds

This testing strategy, on the face of it, sounds exhaustive. If every statement in theprogram is checked for its validity, there does not seem to be much scope of error

## BLACK BOX TESTING

Black box testing is also known as specification testing. To perform black box testing, the analyst examines the specification taking what the program or module should do and how it should perform on the various conditions and submitted for processing. By examine the result, the analyst can examine whether the program performs according to the specified requirements.

**Testing objectives are:** Testing is a process of executing a program with the intendof finding an error.

## TYPES OF TESTING

Different types of testing are:

• Unit testing

• Integration testing

• System testing

• Validation testing

• User acceptance testing

## UNIT TESTING

Unit testing enables a programmer to detect error in coding. A unit test focuses verification of the smallest unit of software design. This testing was carried out during the coding itself. Among the more common errors in computation are:

• Misunderstood or incorrect arithmetic precedence.

• Mixed mode operations.

• Incorrect initialization.

• Precision inaccuracy.

• Incorrect symbolic representation of an expression.

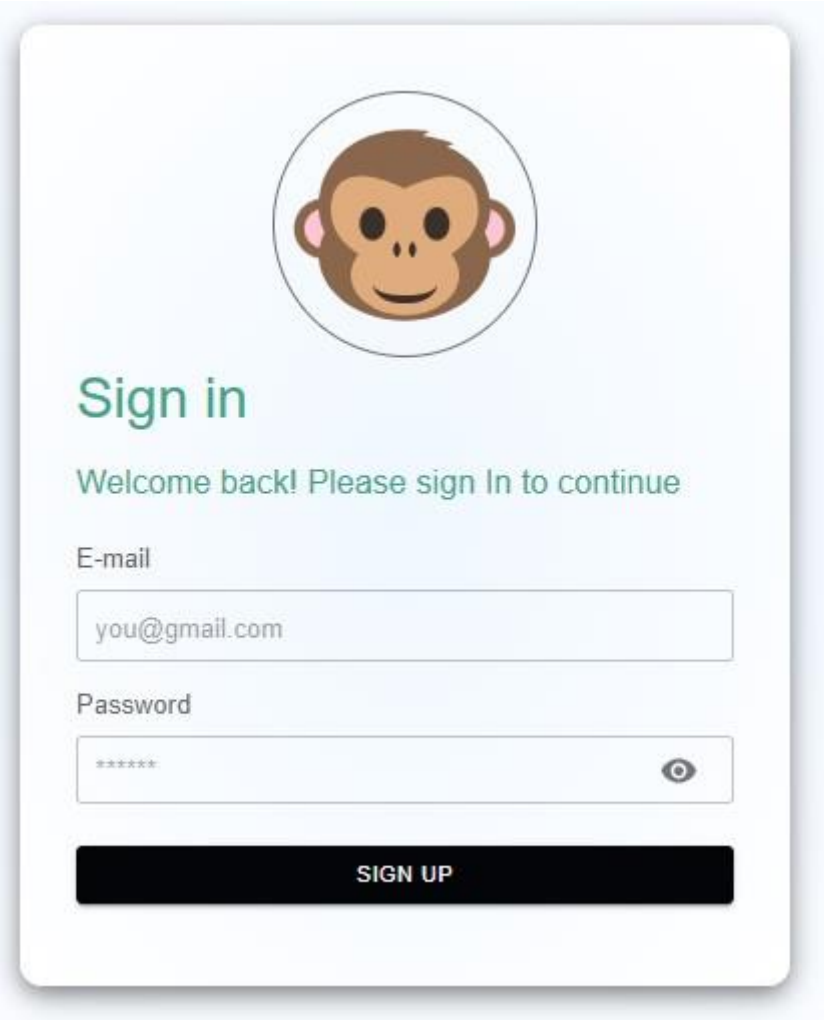
This testing was carried out during the coding itself. In this testing step, each module is going to be work satisfactorily as the expected output from the module

**Project aspect**: The front-end design consists of various forms. They were tested for data acceptance. Similarly, the back-end also tested for successful acceptance and retrieval of data.

The test cases are prepared for each module. Here testing are done in order to verify whether the user can successfully register to the system or not.

The following are some of the test cases for unit testing:

**LOGIN FORM**

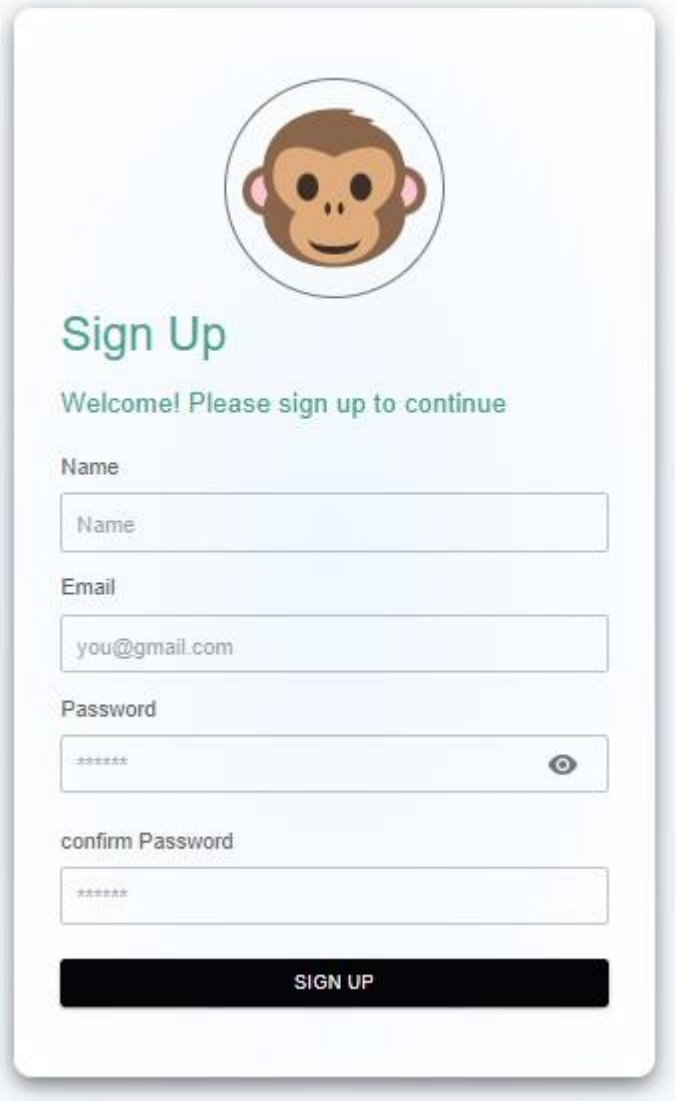


## : Test Cases for Login Form

| Test Case ID | Test Case Description | Expected Results | Actual Results |
|---|---|---|---|
| 1. | Login with valid email and valid password | Login successful and user redirected to respective dashboard | As Expected |
| 2. | Login with empty email field | Validation message “required” is displayed | As Expected |
| 3. | Login with invalid email format | Validation message “invalid E-mail” is displayed | As Expected |
| 4. | Login with empty password field | Validation message “Password is required” is displayed | As Expected |
| 5. | Login with password less than 8 characters | Validation message “Minimum 8 characters” is displayed | As Expected |
| 6. | Login with password without uppercase letter | Validation message “Must contain one uppercase letter” is displayed | As Expected |
| 7. | Login with password without lowercase letter | Validation message “Must contain one lowercase letter” is displayed | As Expected |
| 8. | Login with password without numeric value | Validation message “Must contain one number” is displayed | As Expected |
| 9. | Login with password without special character | Validation message “Must contain one special character” is displayed | As Expected |
| 10. | Login with valid credentials of a **User** | User is redirected to User Dashboard | As Expected |
| 11. | Login with valid credentials of an **Artist** | Artist is redirected to Artist Dashboard | As Expected |
| 12. | Login with valid credentials of an **Admin** | Admin is redirected to Admin Dashboard | As Expected |
| 13. | Login attempt with unregistered user | User is redirected to Signup page | As Expected |

## SIGNUP

| Test Case ID | Test Case Description | Expected Results | Actual Results |
|---|---|---|---|
| 1. | Signup with all valid details | Account is created successfully | As Expected |
| 2. | Signup with empty required fields | Validation messages displayed for required fields | As Expected |
| 3. | Signup with invalid email format | Validation message for invalid email displayed | As Expected |
| 4. | Signup with weak password | Password strength validation error is displayed | As Expected |
| 5. | Signup with password and confirm password mismatch | Error message displayed for password mismatch | As Expected |
| 6. | Signup with already registered email | Error message “Email already exists” displayed | As Expected |
| 7. | Signup as **User** with valid details | User account created and redirected to User Login/Dashboard | As Expected |
| 8. | Backend validation for duplicate email | Duplicate entry rejected and error returned | As Expected |
| 9. | Signup failure due to server error | Proper error message displayed | As Expected |

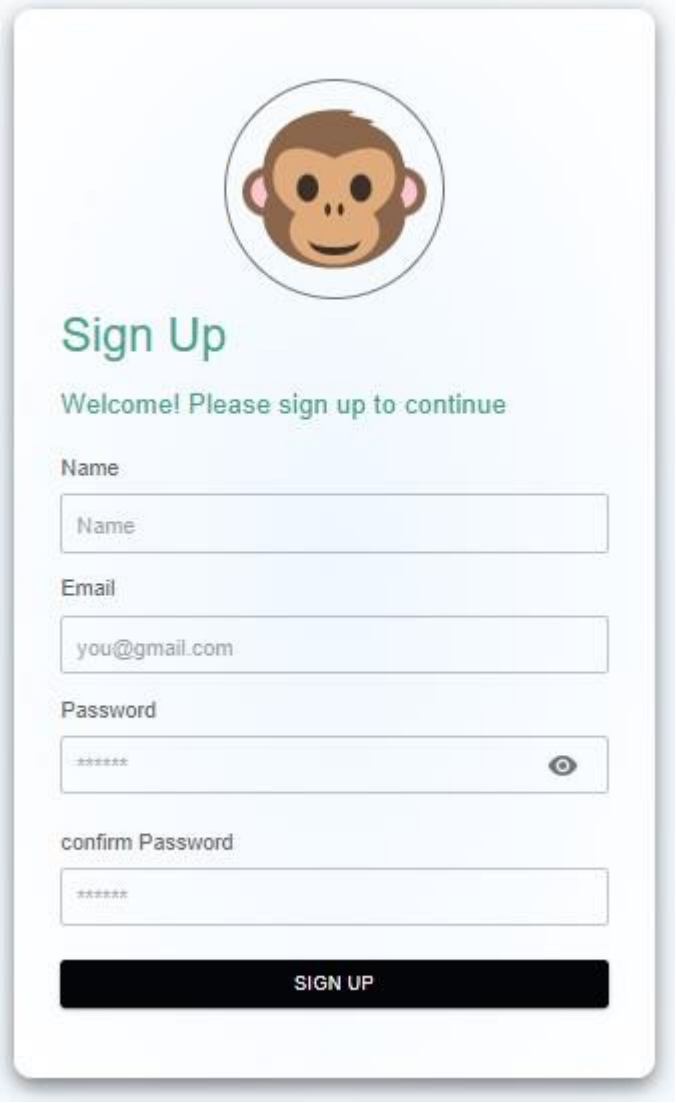



## INTEGRATION TESTING

Through each program work individually, they should work after linking together. This is referred to as interfacing. Data may be lost across the interface; one module can have adverse effect on the other. Subroutines after linking may not do the desired function expected by the main routine. Integration testing is the systematic technique for constructing the program structure while at the same time conducting test to uncover errors associated with the interface. Using integrated test plan prepared in the design phase of the system development as a guide, the integration test wascarried out. All the errors found in the system were corrected for the next testing step.

**Project aspect:** After connecting the back-end and the front-end as whole module, the data entered in the front-end once submitted were successfully entered in the database. On request, data were successfully retrieved in to forms.

## USER REGISTERATION FORM




0


```
_id: ObjectId('6984dd4f94e6328ba1f798c7')
name : "AMINA"
email : "amina1@gmail.com"
password : "$2b$10$v3ygYPlxGyyWN2VTgAmDGuTRgl79KC78wcHPJ2bxa173eRdLEqZva"
role : "user"
isBlocked : false
artistRating : 0
ratingCount : 0
profileInfo : Object
__v : 0
```

## SYSTEM TESTING

After performing the integration testing, the next step is output testing of the proposed system. No system could be useful if it doesn't produce the required output in a specified format. The output generated are displayed by the system under consideration and then tested by comparing with the format require by the user. Here the output format is considered in to two ways, one in on-screen and other in printed format.

The PixelPact platform was tested across multiple environments including web browsers and mobile platforms. The system was evaluated for functionality, performance, security, and usability. All major modules such as user registration, login, artwork creation, blockchain ownership recording, cart and payment processing, chat, and feedback were tested as a complete system.

Special attention was given to role-based access for users, artists, and administrators to ensure correct navigation and data access. Data security and integrity were verified across MongoDB, blockchain storage, and backend services.

## VALIDATION TESTING

The user has to work with the system and check whether the project meets his needs. In the validation checking, the user works with the beta version of the software.

**Project aspects:** In PixelPact, users and artists enter various types of data such as personal details, login credentials, artwork information, and payment details. All inputs are validated at the frontend and backend to ensure correctness, security, and data integrity.

Validation checks are performed on the following fields:

- **Text Fields:**

Text fields such as name, place, artwork title, and description accept only valid characters within the defined length. Alphanumeric validation is applied where required. Invalid or excessive input automatically triggers an error message.

- **Numeric Fields:**

Numeric fields such as phone number, pin code, price, and quantity accept only digits (0–9). Any non-numeric input results in an error message, preventing incorrect data entry.

- **Email Validation:**

The email field is validated to ensure it follows a standard format containing @ and domain extensions such as .com or .in. Incorrect email formats are rejected and appropriate error messages are displayed.

- **Password Validation:**

Password fields are validated to ensure strong security. Passwords must meet minimum length requirements and include uppercase letters, lowercase letters, numbers, and special characters.

- **Form-Level Validation:**

All mandatory fields in signup, login, artwork upload, and payment forms are validated before submission. The system does not allow form submission until all required fields are correctly filled.

## USER ACCEPTANCE TEST

User acceptance of a system is a key factor of the success of any system. The system under consideration was tested for user acceptance by running a prototype of the software.

### Project aspects:

An alpha version of the PixelPact platform was demonstrated to selected users, professional artists, and administrators. Participants interacted with core features such as user registration, AI-based artwork generation, artist collaboration, blockchain-based ownership recording, cart and payment processing, and communication modules. Feedback from users was carefully analyzed and incorporated into the final version of the system.

## Key Feedback Received

- Users appreciated the AI-assisted artwork generation and customization features.
- Artists found the collaboration and commission-based workflow efficient and flexible.
- The blockchain-based ownership verification was valued for protecting originality and preventing duplication.
- Users found the role-based dashboards intuitive and easy to navigate.
- Overall user satisfaction rating: **4.6 / 5**

## Improvements Made Based on User Feedback

- Enhanced UI/UX for easier artwork browsing and customization.
- Improved AI prompt handling for more accurate artwork generation.
- Optimized blockchain recording process for faster ownership confirmation.
- Improved cart and payment workflow for smoother checkout.
- Enhanced chat and notification system for better communication between users and artists.

## WEB TESTING

Web testing is conducted on a web application with an intent to find out potential bugs or defects that could hamper the functionality of that application. During this testing, we also check for Accessibility and Security issues.

**Web testing includes the following types of testing: -**

• Functionality Testing

• Graphical User Interface Testing

• Security Testing

• Database Testing

• Cross Browser/Compatibility Testing

• Performance Testing

• Usability Testing

## FUNCTIONALITY TESTING

Functionality testing is conducted to verify whether the website is developed as per the functional requirements defined in the business documentation or not. Following activities should be performed during functionality testing

1. URL Testing/ Link Testing:- All URLs should be directed towards the appropriate webpage. Project Aspects: Check the URL of admin and user interface There shouldn't be any 404 error page or a broken URL.
2. Form Testing/ Input Field Testing:- All input fields should be tested by putting different types of characters and special characters. There should be proper error messages for mandatory fields, in case a user hasn't filled those.(screen shot email) Project Aspects: Check the field or elements in every page, check the meaningful error messages
3. Cookies Testing:- Cookies are a small set of files used by a web application to store user sessions. Cookies should not store any vital information such as the CVV number of a card for banking transactions.
4. End-to-End Workflow Testing:- In this, business workflow of the web application should be tested. Project Aspects: in our project we tested the connected webpages

## GRAPHICAL USER INTERFACE TESTING

Graphical User Interface testing (GUI testing) is an important phase of web application testing. In this phase of testing, Project Aspects: in our project we check the forms are visible

## SECURITY TESTING

Security testing is a crucial aspect under web testing, especially for the banking domain and the e-commerce websites as these websites process vital user information such as bank account details, card information, passwords, etc.

**Project Aspects:** in our project we check the authentication with username and password.

## DATABASE TESTING

In Database testing, we mainly test for errors (if any) at the time of queries' execution and the response time of those queries' execution. In such a type of testing, we also check the flow of data, i.e., to test data fetched from the database to ensure accurate visibility in the web application.

Project Aspects: in our project we checked different data manipulation commands (add, update, delete) and data integrity functionality

## CROSS BROWSER / COMPATIBILITY TESTING

In this, testing is done for a web application against different combinations of browsers and operating systems such as-Firefox with Win7, Safari with Mac OS, etc. Intent of this testing is to verify that the web application behaves consistently acrossall OS's and browsers wherein there should not be any browser or OS specific bug.

**Project Aspects:** In our project we checked in different browsers like brave, Mozillaand chrome etc.

## PERFOMANCE TESTING

In this, the response time of a web application and its behaviour during peak and nonpeak load traffic is tested. It is to check an application's recovery time when it crashes at a certain point of time.

**Project Aspects:** in our project we checked the behaviour of the system during peak and non-peak time

## USABILITY TESTING

Usability testing has also become an important part of web application testing. This contains testing for web elements such as-Forms, Text boxes, Images, Menu items, Links etc., ensuring its consistency and visibility across all web pages. This type of testing is also conducted to ensure that there aren't any spelling or grammatical mistakes in the web page and that every image has an alternate text.

**Project Aspects:** In our project we checked system navigation, alignment of elements, confirmation of images etc.

# 8

# IMPLEMENTATION

## Introduction

The implementation phase of PixelPact focuses on making the system accessible to users, artists, and administrators through both web and mobile platforms. After successful development and testing, the PixelPact website and mobile application are made available for public use. Users can access the platform either through a web browser or by downloading the mobile application.

The system ensures smooth interaction between AI-based artwork generation, artist collaboration, blockchain ownership verification, and secure payment processing.

## 1. Accessing the PixelPact Website

PixelPact can be accessed through any modern web browser such as Chrome, Edge, or Firefox.

- Users open the official PixelPact website URL.
- The homepage displays options such as Register, Login, Explore Artists, and Generate Art.
- New users can create an account.
- Existing users can log in using their credentials.
- After login, users are redirected to their respective dashboards (User, Artist, or Admin).

The website provides full access to features such as AI image generation, artist collaboration, viewing artworks, sending requests, payments, and blockchain ownership verification.

## 2. Downloading and Installing the PixelPact Mobile App

PixelPact also provides a mobile application developed using Flutter for Android (and iOS if supported).

Steps to use the mobile app:

1. Download the PixelPact app from the official distribution platform.
2. Install the application on the mobile device.
3. Open the application and register or log in.
4. After login, access features similar to the web platform.

The mobile app offers a user-friendly interface optimized for smartphones, allowing users to generate images, chat with artists, send work requests, and track payment status conveniently.

### 3. User Interaction Flow

After logging in, users can:

- Generate AI-based artwork by entering prompts and style preferences.
- Upload reference images for customization.
- View available artists and send commission requests.
- Make secure payments for artwork.
- Verify blockchain ownership of finalized artwork.
- Provide ratings and feedback.

### 4. Artist Interaction Flow

Artists can:

- Register and complete profile verification.
- Upload and manage artwork portfolios.
- Chat with users for clarification.
- Receive payments after successful completion.
- View ratings and reviews.

.

### 5 Admin Access

. The Admin logs into the system through a secure panel to:

- Verify artist accounts.
- Monitor users and artworks.

- Handle complaints.
- Block or unblock accounts if necessary.
- Supervise overall platform activity.

**6. Live Operation and Availability**

Once deployed, PixelPact operates in a live environment where:

- The website is available 24/7.
- The mobile app provides continuous access.
- AI services generate artwork in real-time.
- Blockchain records ensure secure ownership tracking.
- Payments are processed securely

This implementation approach ensures that PixelPact is easily accessible, user-friendly, secure, and fully operational across both web and mobile platforms.

# 9

# CONCLUSION AND FUTURE ENHANCEMENT

# FUTURE SCOPE OF THE PROJECT

The PixelPact platform has significant potential for expansion and enhancement as digital art, AI, and blockchain technologies continue to evolve. Some of the key future scopes of this project are outlined below:

- **Advanced AI and Personalization Enhancements-** Future versions of PixelPact can integrate more advanced generative AI models to produce higher-quality, more detailed, and emotionally expressive artworks. AI can be trained to better understand complex prompts, cultural art styles, and user preferences. Personalized AI recommendations based on user history, trending styles, and artist performance can further improve user satisfaction and creativity.
- **NFT Integration and Digital Art Marketplace Expansion-** Pi xelPact can be expanded into a full-fledged NFT-enabled digital art marketplace. By minting artworks as NFTs directly on the platform, users and artists can buy, sell, and auction digital art securely. Smart contracts can automate royalty payments to artists on every resale, ensuring long-term income and fair compensation for creators.
- **Augmented Reality and Immersive Art Experiences-** The platform can be extended to support Augmented Reality (AR) features, allowing users to preview artworks in real-world environments such as walls, galleries, or exhibitions. Virtual art galleries and metaverse integration can provide immersive experiences for showcasing and collaborating on digital artwork globally.
- **Mobile, Multi-Platform, and Cloud Scalability-** Future development may focus on optimizing PixelPact for large-scale usage through cloud-based infrastructure and cross-platform support, including web, mobile, and desktop applications. This will enable seamless access, faster rendering, and real-time collaboration between artists and users worldwide.

# CONCLUSION

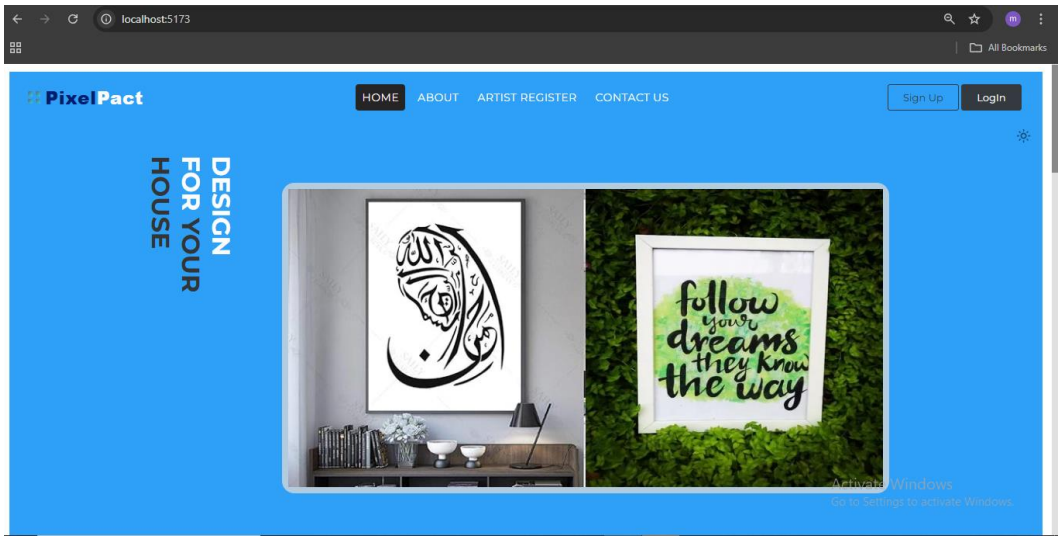
PixelPact presents an innovative approach to digital art creation by successfully combining artificial intelligence, professional artist collaboration, and blockchain-based ownership protection within a single platform. The system addresses critical challenges such as artwork duplication, unauthorized copying, and intellectual property disputes while offering users a personalized and efficient creative experience. By enabling AI-assisted generation alongside human artistic refinement, PixelPact ensures both speed and authenticity in digital art production. With its strong foundation and wide future expansion possibilities, PixelPact has the potential to become a secure, scalable, and trustworthy ecosystem for next-generation digital art creation and collaboration

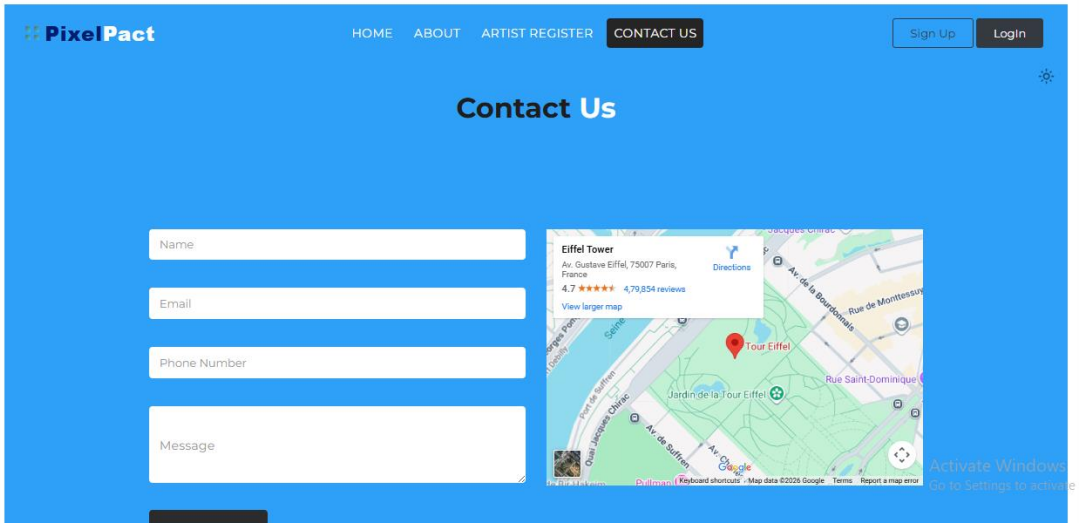
# 10

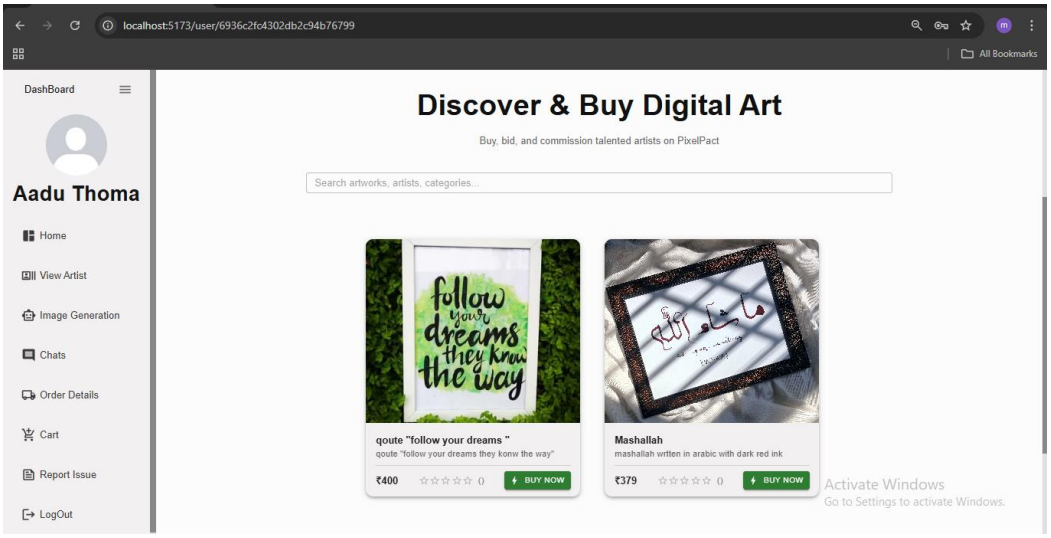
# APPENDIX

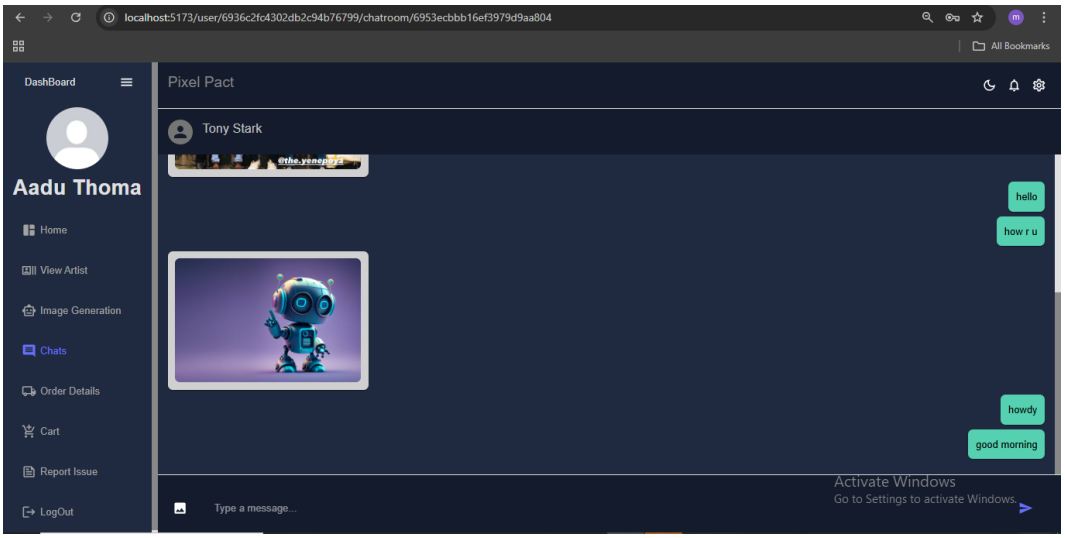
# SCREENSHOT

## SCREEN LAYOUTS WEB PAGES



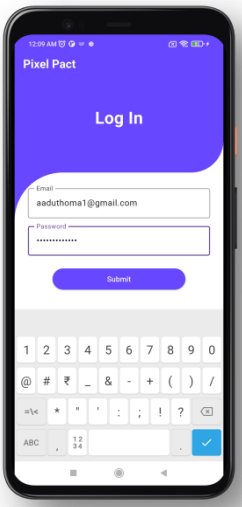



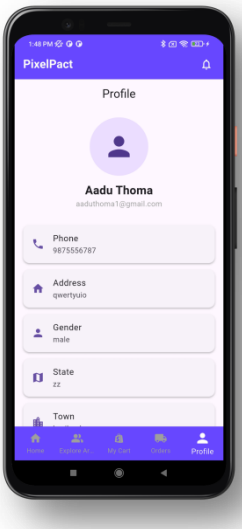



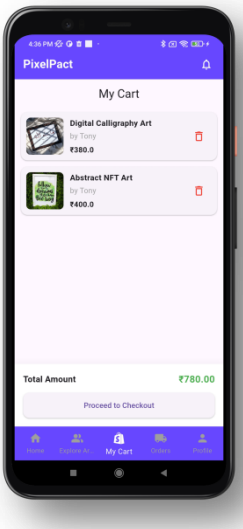

localhost:5173/user/6936c2fc4302db2c94b76799/chatroom/6953ecbbb16ef3979d9aa804
All Bookmarks
DashBoard
Pixel Pact
Aadu Thoma
Home
View Artist
Image Generation
Chats
Order Details
Cart
Report Issue
LogOut
Tony Stark
hello
how r u
howdy
good morning
Type a message...
Activate Windows
Go to Settings to activate Windows.

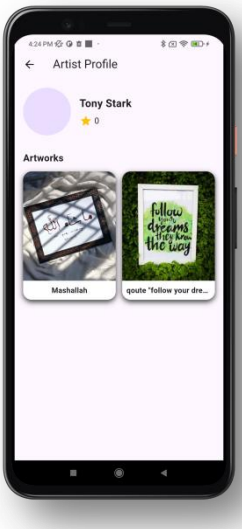
Pixel Pact
DashBoard
Muhammed Niyad
Home
View Artist
New Artist
Comlpaints
Payments
Veiw User
LogOut
Admin Dashboard
Overview & quick actions
TOTAL USERS
4
ARTISTS
1
COMPLAINTS
2
REVENUE
₹2.4L
PLATFORM GROWTH
complaints
Nov
Month
USER ROLES
Artists
Users
MONTHLY COMPLAINTS
Rating
12
Tony Stark
Name

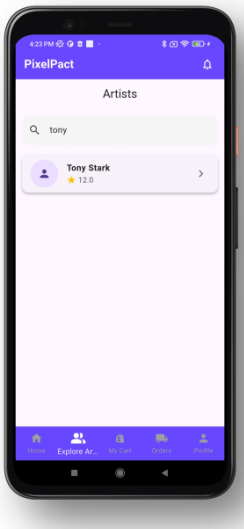
# SCREEN LAYOUT ANDROID



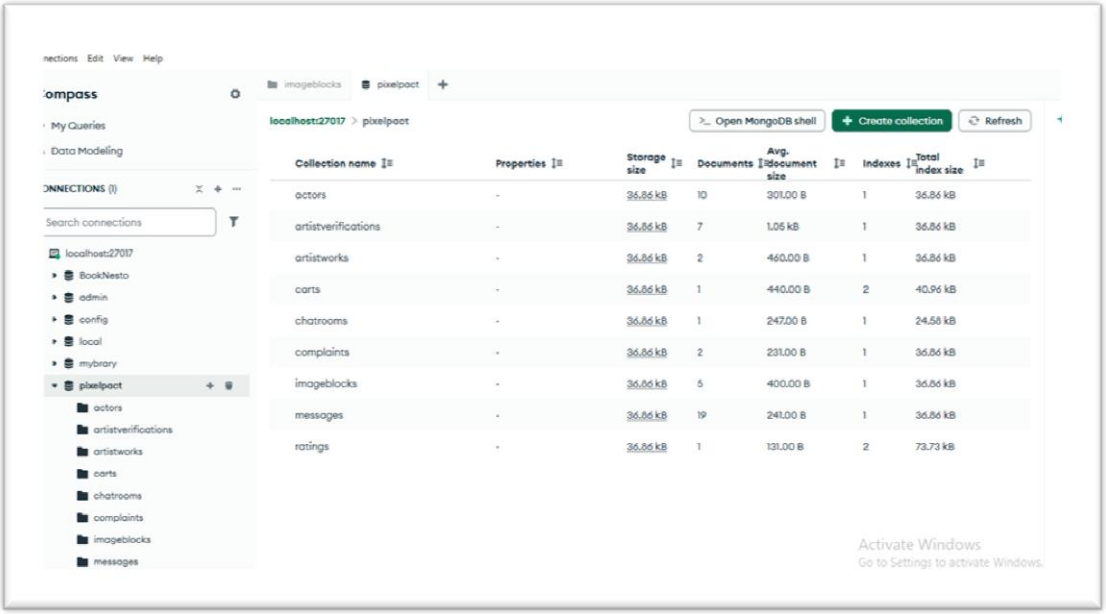


# DATABASE




## Blockchain:Image-Block


```
{
  "_id": {
    "$oid": "693ec046c3b50d63609b83c8"
  },
  "index": 1,
  "timestamp": 1765720134425,
  "data": {
    "imageHash": "3b2c948ad080b47c13f096e3283bb01a88e881d4f1608703faf40c3bcf9916e1",
    "filename": "1765720133860_photo.jpg",
    "path": "uploads\\Artwork_images\\1765720133860_photo.jpg",
    "uploadedAt": {
      "$date": "2025-12-14T13:48:54.019Z"
    }
  },
  "previousHash": "dd75796f4704ee63db7b0518140c23a90b1fb15ca5bd8b553acfd75dd7be8c07",
  "hash": "c8bfe3b7273e739315eb6fccae2ff3de58a5ef251588ecde26eaeca41e593478",
  "__v": 0
}
```

**Chatroom :record**


```
_id: ObjectId('6953ecbbb16ef3979d9aa804')
artistId : ObjectId('692973c1c337cdeda52f2e2a')
userId : ObjectId('6936c2fc4302db2c94b76799')
artistName : "Tony Stark"
userName : "Aadu Thoma"
userProfile : ""
artistProfile : ""
createdAt : 2025-12-30T15:16:11.761+00:00
updatedAt : 2026-02-09T16:48:12.604+00:00
__v : 0
lastMessage : "good morning"
lastMessageAt : 2026-02-09T16:48:12.604+00:00
```


**Message Record**


```
_id: ObjectId('695e7d1f7f06cfe8534221c6')
chatroomId : ObjectId('6936c2fc4302db2c94b76799')
senderId : ObjectId('6936c2fc4302db2c94b76799')
messageType : "text"
role : "user"
text : "A dark brow frame containing an islamic calligraphy art work with blue…"
seen : false
createdAt : 2026-01-07T15:34:55.104+00:00
updatedAt : 2026-01-07T15:34:55.104+00:00
__v : 0
```


**Admin Record**


```
_id: ObjectId('6922fd63a5cec3474fb71236')
name : "Muhammed Niyad"
email : "muhammedniyad720@gmail.com"
password : "$2b$10$jZTDAkO3fQhkwFh4Z0Sg9.m6yJKeXgAsQgqLkxYZZyKdCyl/DNDTC"
role : "admin"
▾ profileInfo : Object
    dob : 2004-10-03T18:30:00.000+00:00
    gender : "male"
    phoneNo : "8606079774"
    state : "kerala"
    town : "koyilandy"
    pin : 650533
    address : "Koyilandy,mubarak road,mabrook"
    profileImg : "1764245507187_Grey-hat.jpg"
```

# 11

# REFERENCES

**BOOKS**

[1] K. Chodorow, *MongoDB: The Definitive Guide*, 3rd ed. Sebastopol, CA, USA: O'Reilly Media, 2019.

[2] E. Brown, *Web Development with Node and Express: Leveraging the JavaScript Stack*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2019.

[3] A. Banks and E. Porcello, *React: Modern Patterns for Developing UIs*. Sebastopol, CA, USA: O'Reilly Media, 2020.

[4] M. Casciaro and L. Mammino, *Node.js Design Patterns*, 3rd ed. Birmingham, U.K.: Packt Publishing, 2020.

[5] S. Ramírez, *FastAPI Documentation and Conceptual Guide*. [Online]. Available: https://fastapi.tiangolo.com/

[6] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Upper Saddle River, NJ, USA: Pearson, 2020.

[7] I. Bashir, *Mastering Blockchain: Distributed Ledger Technology, Cryptography, and Smart Contracts*, 3rd ed. Birmingham, U.K.: Packt Publishing, 2020.

**WEBSITES**

[8] MongoDB Inc., "MongoDB Official Documentation." [Online]. Available: https://www.mongodb.com/docs

[9] Express.js, "Express – Node.js Web Application Framework." [Online]. Available: https://expressjs.com/

[10] Meta Platforms, Inc., "React Official Documentation." [Online]. Available: https://reactjs.org/docs/getting-started.html

[11] OpenJS Foundation, "Node.js Official Documentation." [Online]. Available: https://nodejs.org/en/docs/

[12] S. Ramírez, "FastAPI Official Documentation." [Online]. Available: https://fastapi.tiangolo.com/

[13] Towards AI, "Towards AI – Artificial Intelligence Resources." [Online]. Available: https://www.towardsai.net/